上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



Data Mining and Big Data Anlysis
数据挖掘及大数据分析

— Chapter 7： Classification分类
7-1：基本概念

Wang Dong 王东

School of Software, Shanghai Jiaotong University

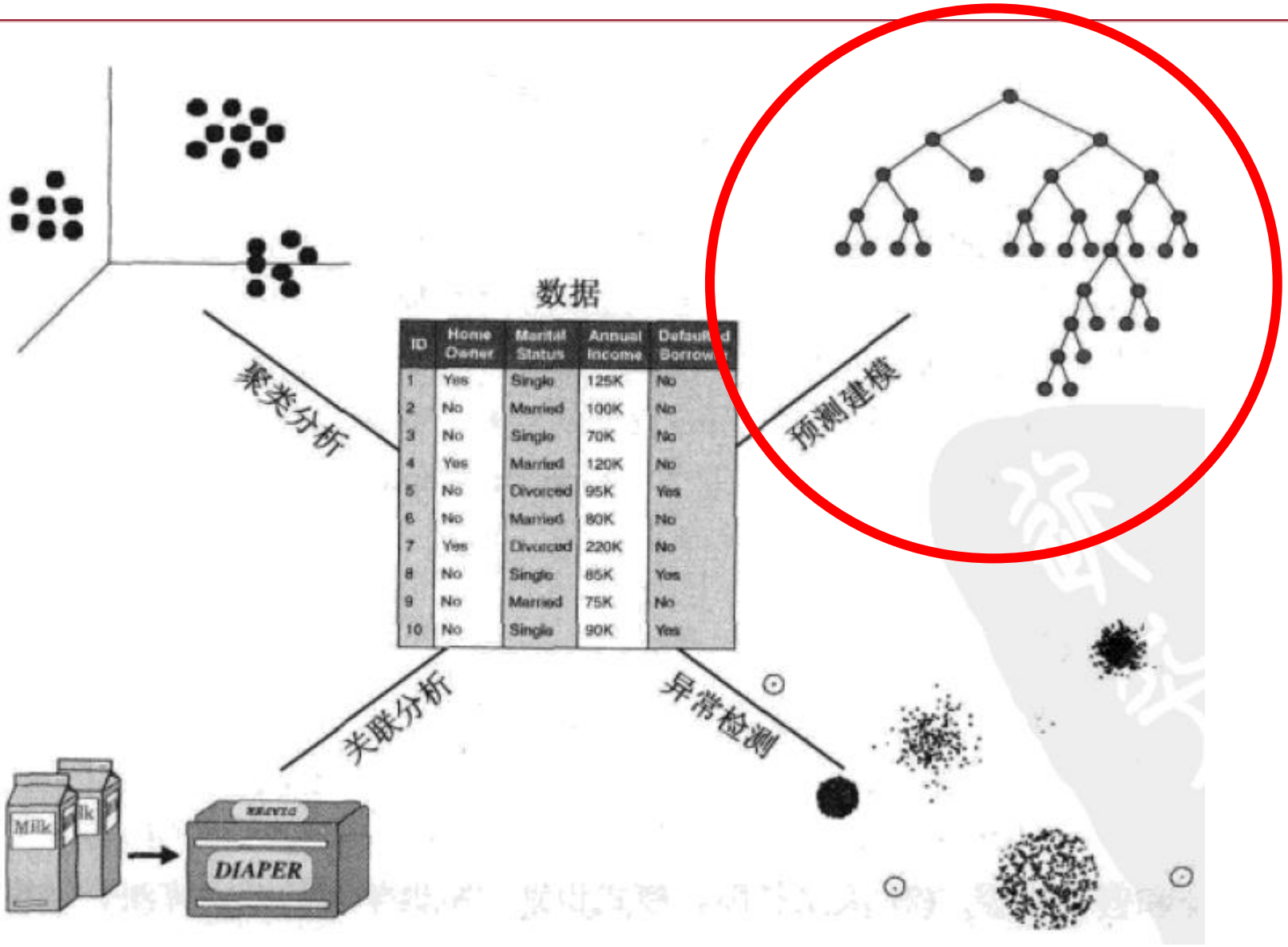
2025年春季

Chapter 7 Classification

第七讲 分类

内容设计

Data Mining Function
4种经典数据挖掘模式



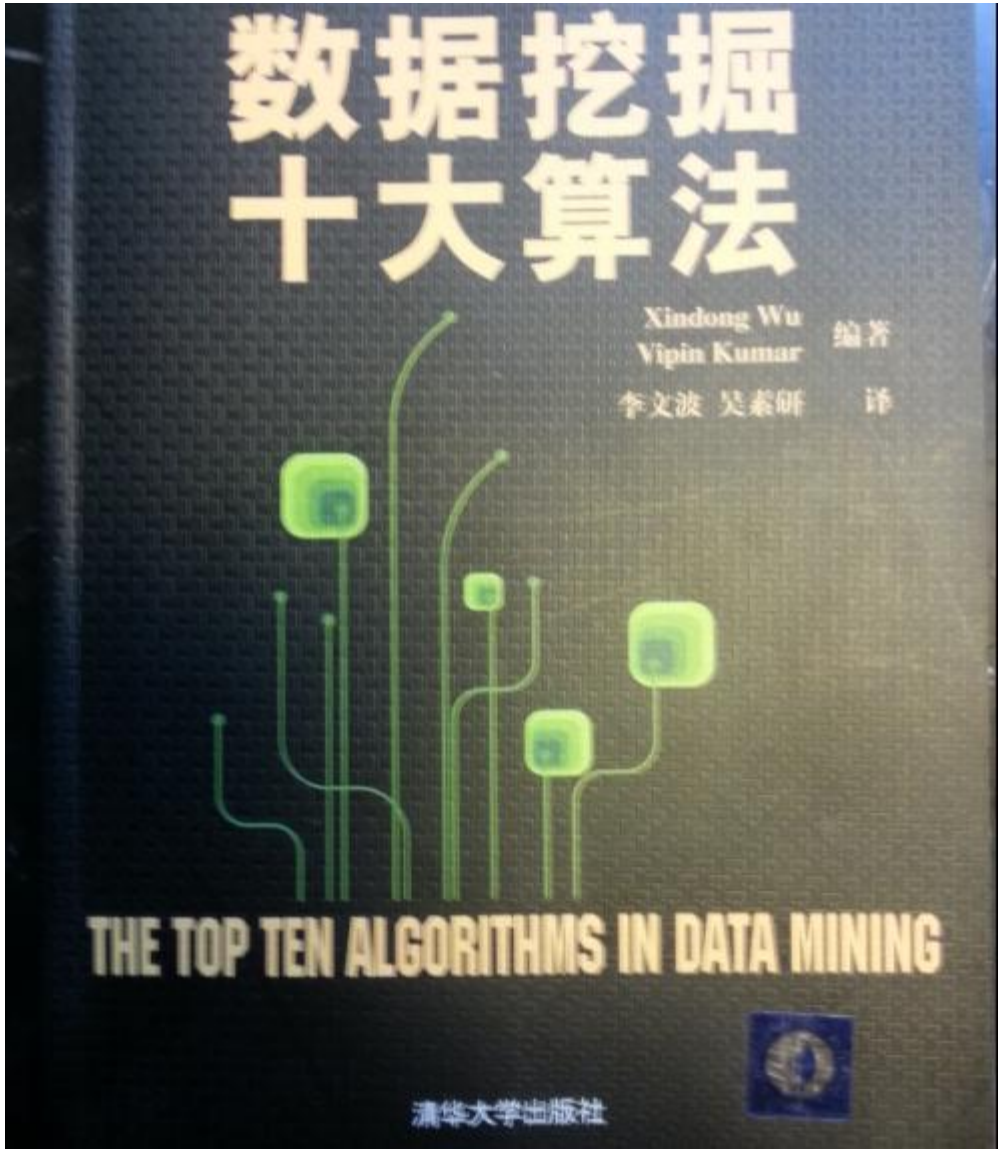
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	80K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Classification Techniques

- Base Classifiers
  - —Decision Tree based Methods
  - —Rule-based Methods
  - —Nearest-neighbor
  - —Naïve Bayes
  - —Bayesian Belief Networks
  - —Support Vector Machines
  - —Neural Networks
  - —Deep Neural Nets
- Ensemble Classifiers 集成分类器
  - —Boosting提升
  - —Bagging装袋
  - —Random Forests随机森林

数据挖掘的“十大”最具影响力的算法

- C4.5
- k-means
- SVM：支持向量机
- Apriori
- EM：期望最大化
- PageRank
- AdaBoost
- KNN：k-最近邻
- Naïve Bayes
- CART：分类和回归树



Chapter 7 Classification 分类
7-1: Basic Concepts 基本概念

- Classification: Basic Concepts 基本概念
- Decision Tree Classifier *决策树分类器(ML 课程)*
- Model Overfitting 模型的过拟合
- Model Selection 模型选择
- Model Evaluation 模型评估
- Presence of Hyper-parameters 超参数的使用
- Pitfalls of Model Selection and Evaluation 模型选择与评估中的陷阱
- Model Comparison 模型比较
- Techniques to Improve Classification Accuracy 提高分类准确率的技术

Chapter 7 Classification 分类
7-2：Adanced Methods 高级方法

- Rule-Based Classification 基于规则的分类
- Bayes Classification Methods *贝叶斯分类(ML课程)*
- Bayesian Belief Networks 贝叶斯信念网络
- Classification by Backpropagation *用后向传播分类(ML课程)*
- Support Vector Machines *支持向量机(ML课程)*
- Classification Using Frequent Patterns 使用频繁模式分类
- Lazy Learners 惰性学习法*(ML课程)*

  (or Learning from Your Neighbors) （或从近邻学习）
- Other Classification Methods 其他分类方法
- Ensemble Learning 集成分类法
- Additional Topics Regarding Classification 关于分类的其它问题



开始“分类”数据挖掘方法之旅

Chapter 7 Classification 分类
7-1: Basic Concepts 基本概念

- **Classification: Basic Concepts** **基本概念**
- Decision Tree Induction 决策树归纳
- Model Overfitting 模型的过拟合
- Model Selection 模型选择
- Model Evaluation 模型评估
- Presence of Hyper-parameters 超参数的使用
- Pitfalls of Model Selection and Evaluation 模型选择与评估中的陷阱
- Model Comparison 模型比较
- Techniques to Improve Classification Accuracy 提高分类准确率的技术

Classification: Definition

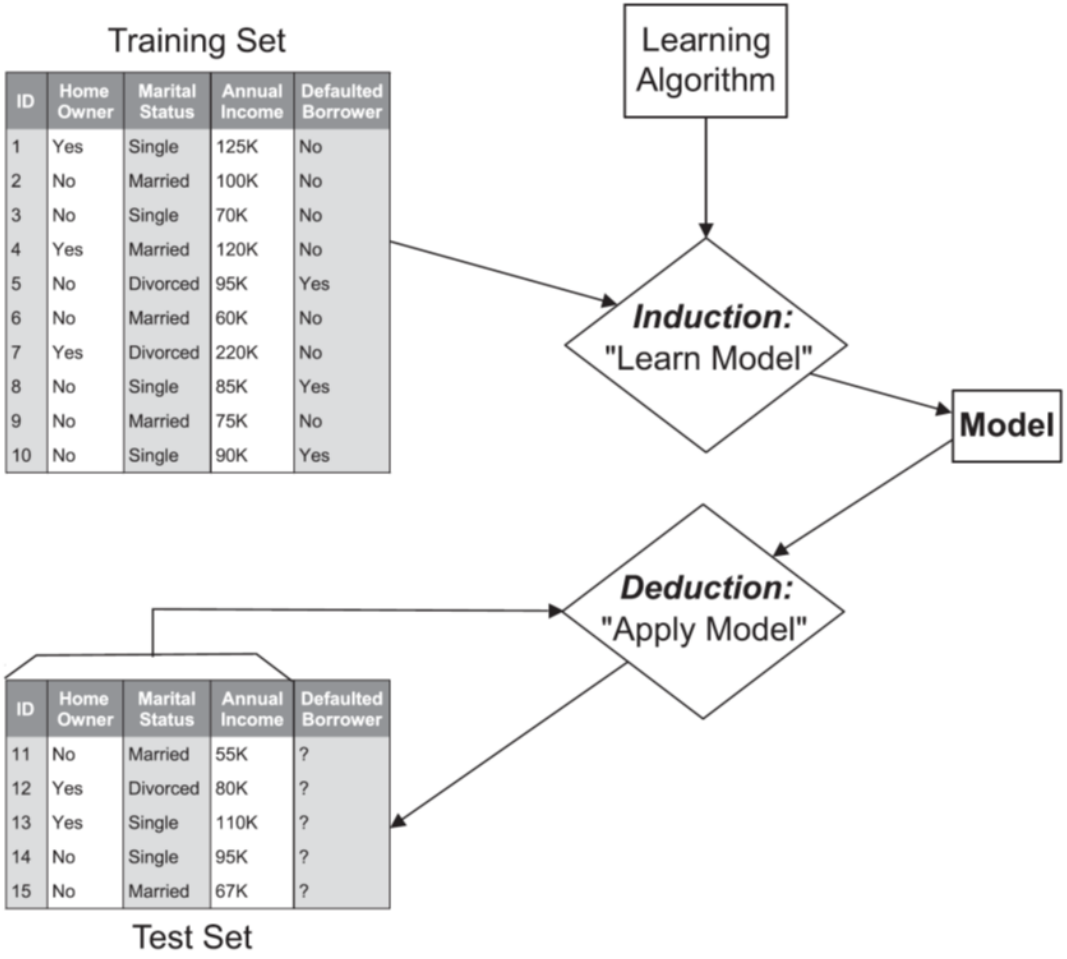
- Given a collection of records (training set)
  - Each record is by characterized by a tuple元组 (x,y), where x is the attribute set and y is the class label
    - x: attribute, predictor, independent variable, input
    - y: class, response, dependent variable, output

- Task:
  - Learn a model that maps each attribute set x into one of the predefined class labels y

Examples of Classification Task

Task	Attribute set, $\boldsymbol{x}$	Class label, y
Categorizing email messages	Features extracted from email message header and content	Spam垃圾邮件 or non-spam
Identifying tumor cells肿瘤	Features extracted from x-rays or MRI scans磁共振成像	malignant or benign cells恶性的[məˈlɪgnənt] 良性的[bɪˈnaɪn]
Cataloging galaxies	Features extracted from telescope images	Elliptical, spiral, or irregular-shaped galaxies 椭圆形[ɪˈlɪptɪkl] 螺旋形的[ˈspaɪrəl]

General Approach for Building Classification Model



ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
11	No	Married	55K	?
12	Yes	Divorced	80K	?
13	Yes	Single	110K	?
14	No	Single	95K	?
15	No	Married	67K	?

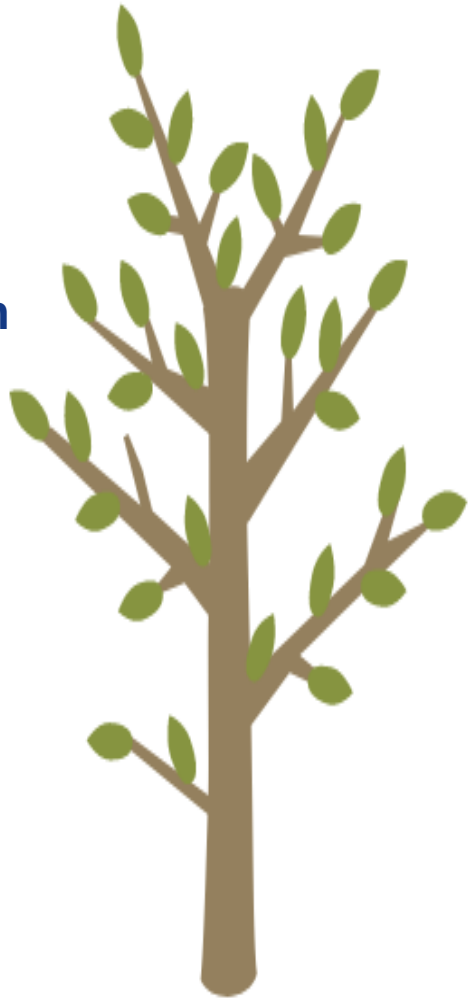
General framework for building a classification model.

Chapter 7 Classification 分类
7-1: Basic Concepts 基本概念

- Classification: Basic Concepts 基本概念
- **Decision Tree Classifier 决策树分类器**
- Model Overfitting 模型的过拟合
- Model Selection 模型选择
- Model Evaluation 模型评估
- Presence of Hyper-parameters 超参数的使用
- Pitfalls of Model Selection and Evaluation 模型选择与评估中的陷阱
- Model Comparison 模型比较
- Techniques to Improve Classification Accuracy 提高分类准确率的技术

Decision Tree Classifier

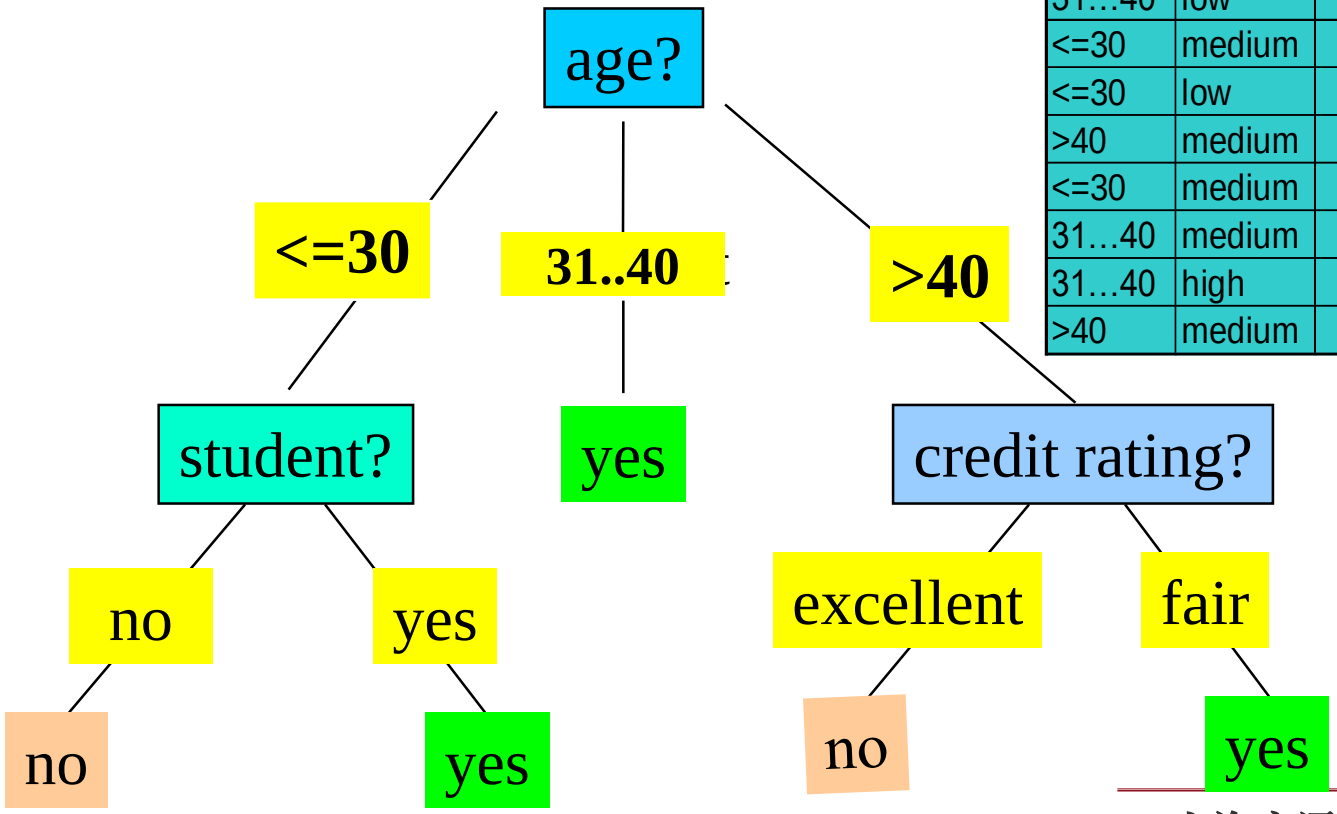
- Overview
- Basic Algorithm to Build a Decision Tree
- Methods for Expressing Attribute Test Conditions
- Measures for Selecting an Attribute Test Condition
- Algorithm for Decision Tree Induction
- Classic decision tree algorithm
- Example Application: Web Robot Detection
- Characteristics of Decision Tree Classifiers



Decision Tree 决策树

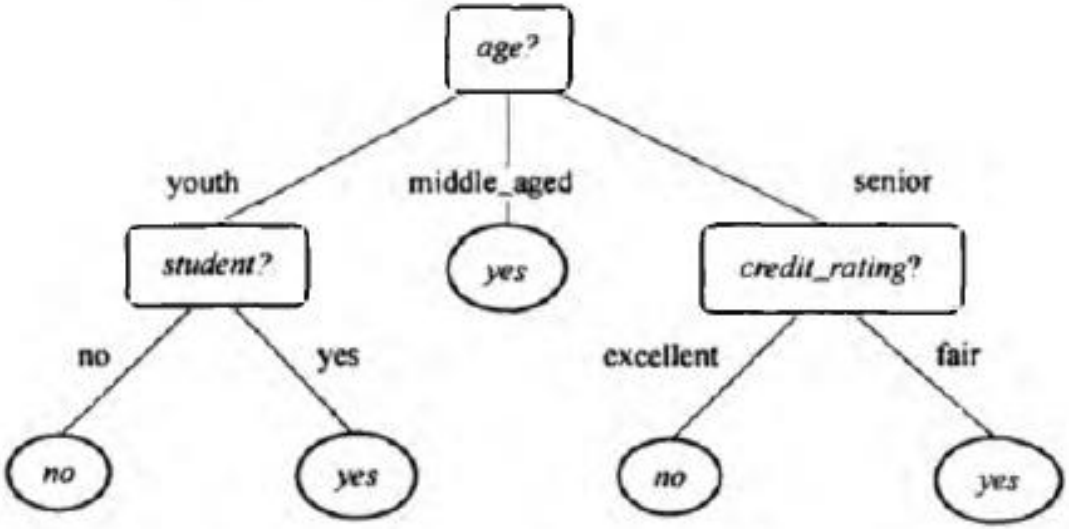
- 目的：对未知样本进行分类
  - 将样本的属性值与决策树相比较
- E.g., Buys_computer的决策树:

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no



决策树归纳的概念

- **决策树归纳**是从有类标号的训练元组中学习决策树
- 决策树是一种类似于流程图的树结构
  - 每个**内部结点**（非树叶结点）表示在一个属性上的测试
  - 每个**分枝**代表该测试的一个输出
  - 每个**树叶结点**（或终端结点）存放一个类标号
  - 树的最顶层结点是**根结点**
- 内部结点用矩形表示，而叶结点用椭圆（或圆角矩形）表示
- 有些决策树算法只产生二叉树（其中，每个内部结点正好分叉出两个其他结点），另一些决策树算法可能产生非二叉的树



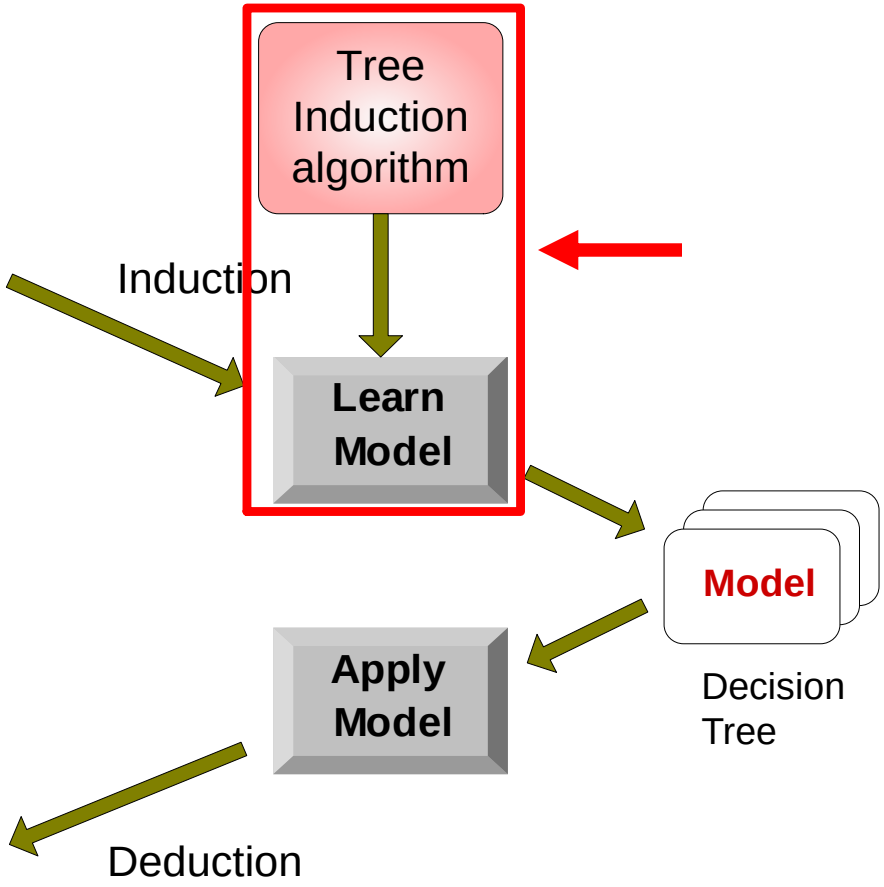
Decision Tree Classification Task

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

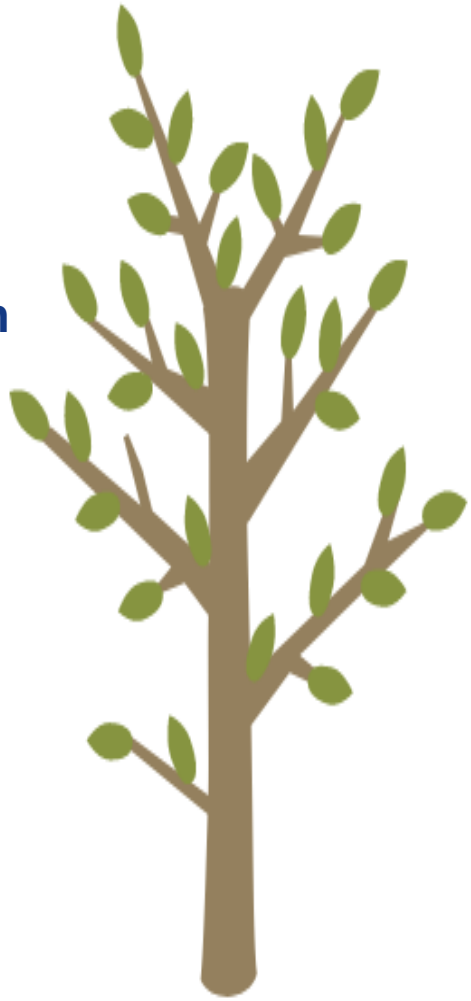
Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



Decision Tree Classifier

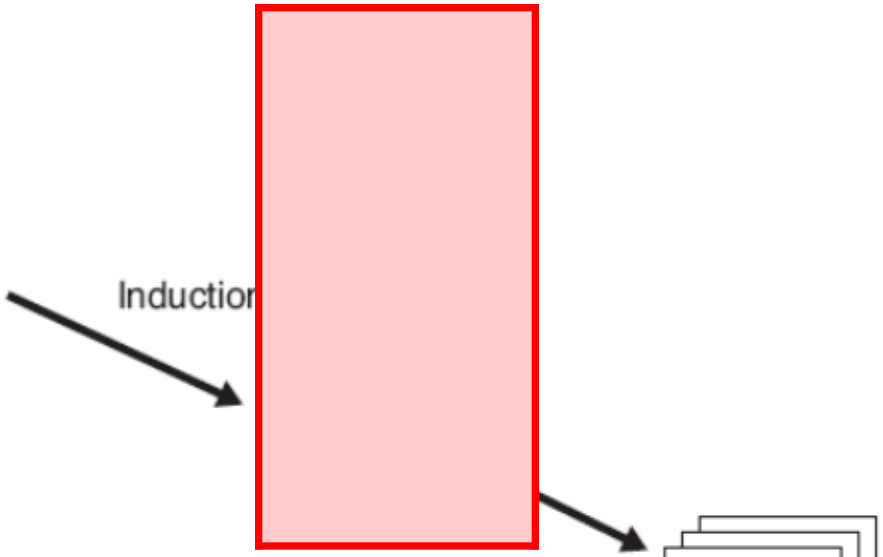
- Overview
- Basic Algorithm to Build a Decision Tree
- Methods for Expressing Attribute Test Conditions
- Measures for Selecting an Attribute Test Condition
- Algorithm for Decision Tree Induction
- Classic decision tree algorithm
- Example Application: Web Robot Detection
- Characteristics of Decision Tree Classifiers



Decision Tree: Induction

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes



Test Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Decision Tree Induction

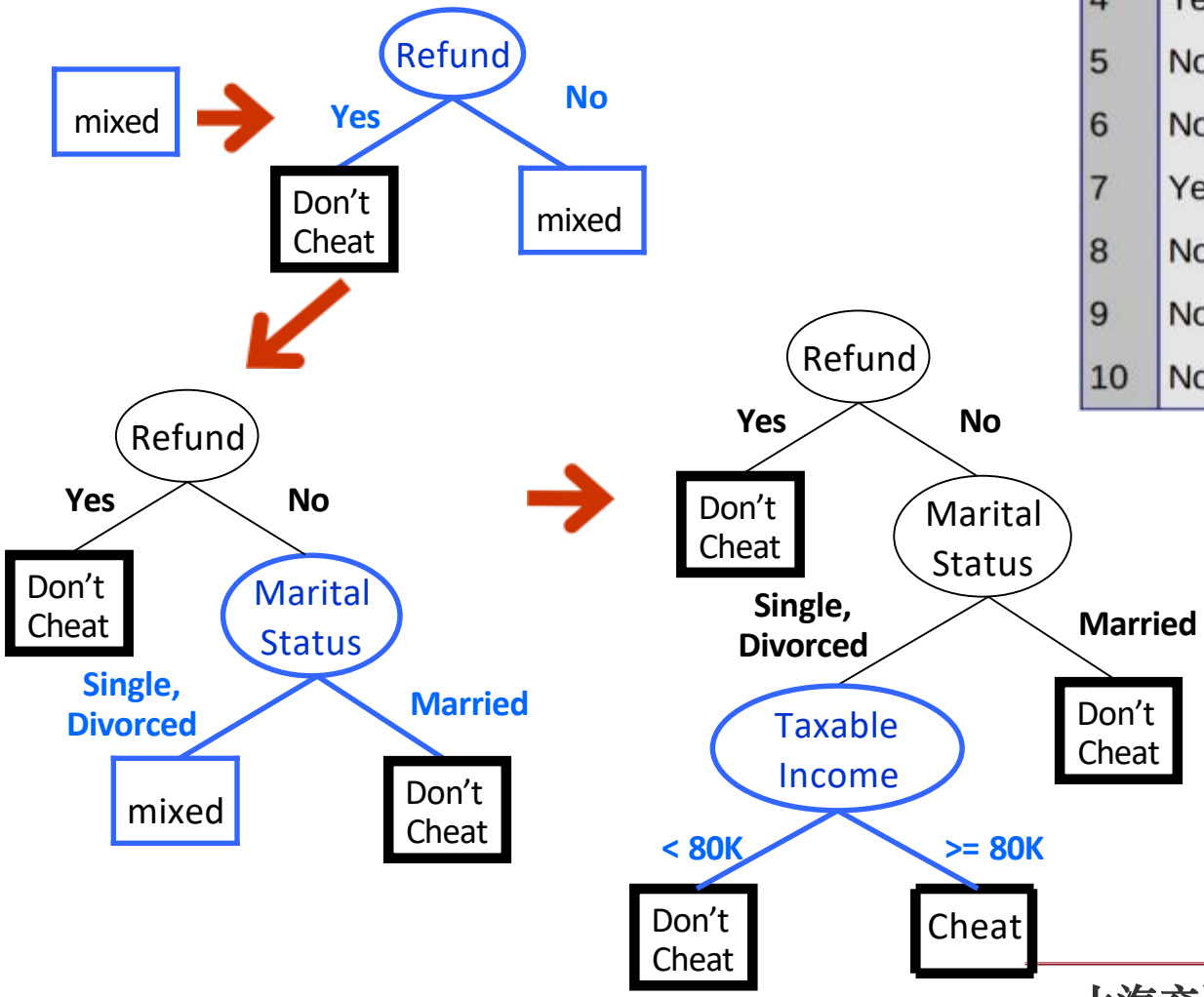
Many Algorithms:

- **Hunt's Algorithm (one of the earliest)**
- **CART (Classification And Regression Tree)**
- **ID3, C4.5, C5.0 (by Ross Quinlan, information gain)**
- **CHAID (CHi-squared Automatic Interaction Detection)**
- **MARS (Improvement for numerical features)**
- **SLIQ, SPRINT**
- **Conditional Inference Trees (recursive partitioning using statistical tests)**

Hunt's Algorithm

"Use attributes to split the data recursively, till each split contains only a single class."

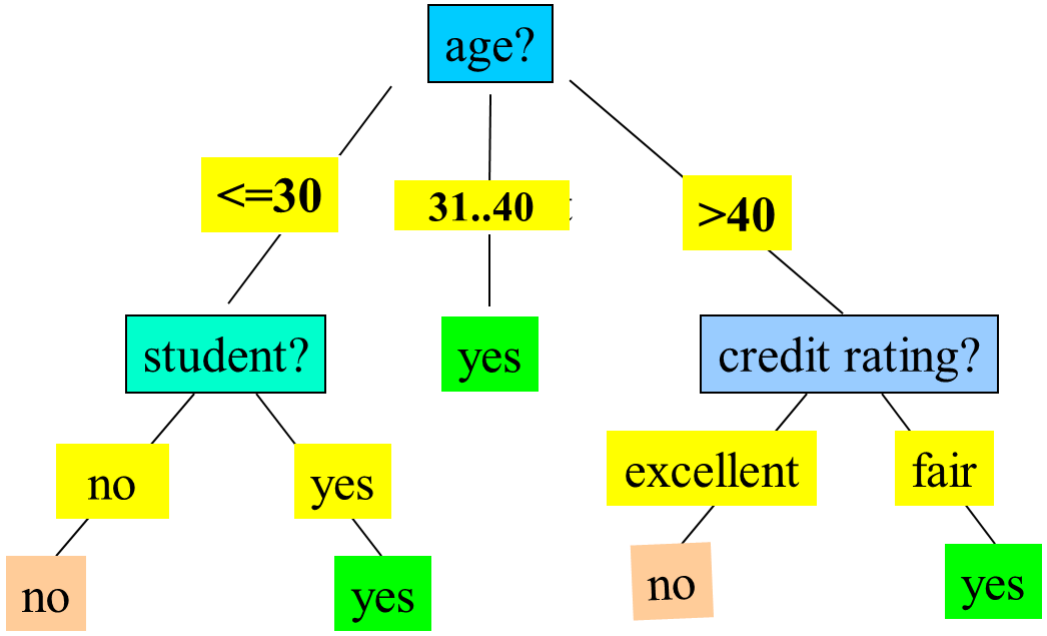
Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



['ri:fʌnd]偿还

决策树归纳（决策树构建）的关键问题

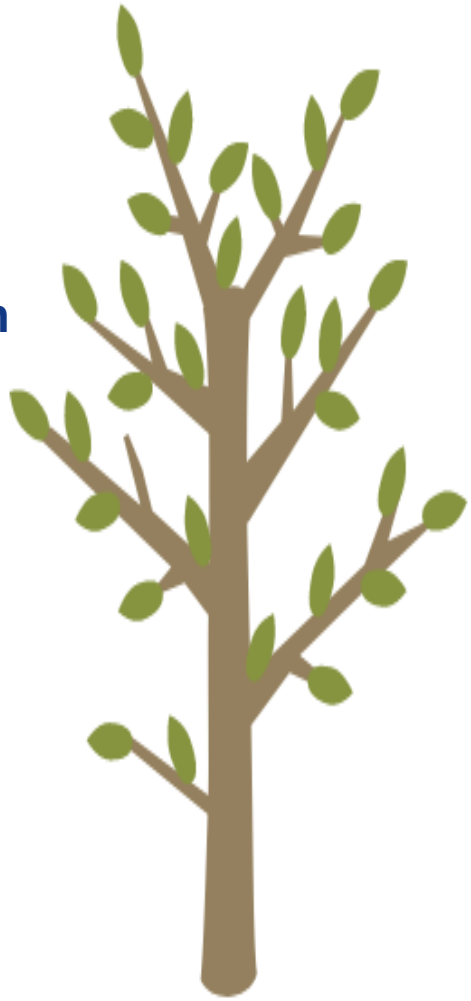
- 决策树构建过程中需要解决三个关键问题
  - **Determine how to split the record using different attribute types** 如何表示属性测试条件
  - **How to determine the best split?**如何选择合适的属性（分裂属性）作为决策树的节点去划分训练样本
  - **Determine when to stop splitting**如何在适当位置停止划分过程，从而得到大小合适的决策树



age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

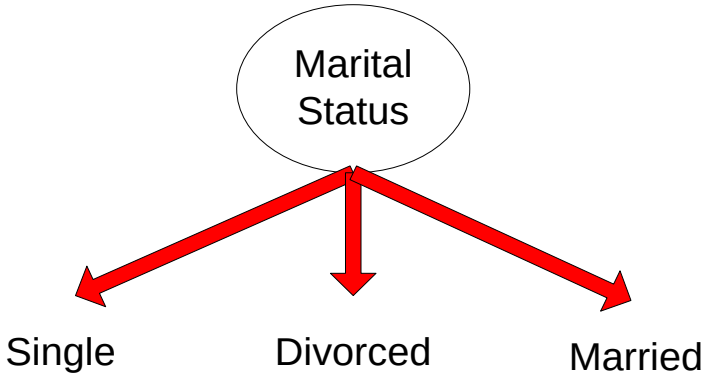
Decision Tree Classifier

- Overview
- Basic Algorithm to Build a Decision Tree
- Methods for Expressing Attribute Test Conditions
- Measures for Selecting an Attribute Test Condition
- Algorithm for Decision Tree Induction
- Classic decision tree algorithm
- Example Application: Web Robot Detection
- Characteristics of Decision Tree Classifiers

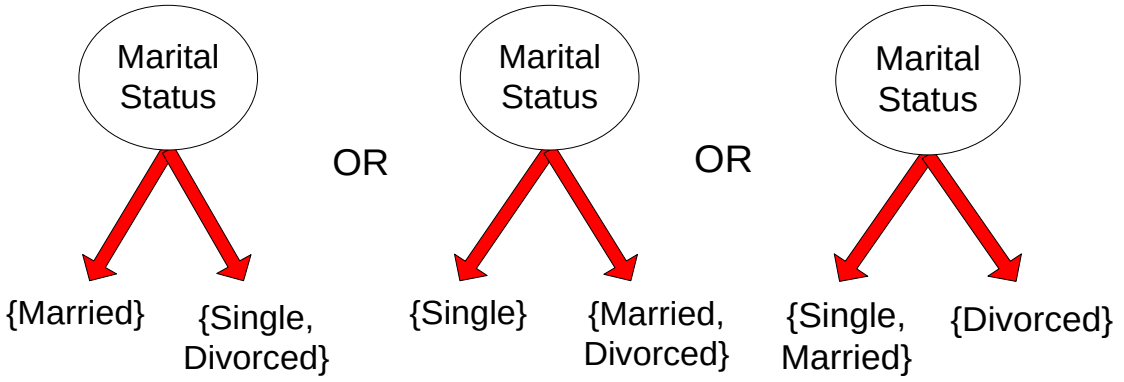


Test Condition for Nominal Attributes

- **Multi-way split**
  - **Use as many partitions as distinct values.**

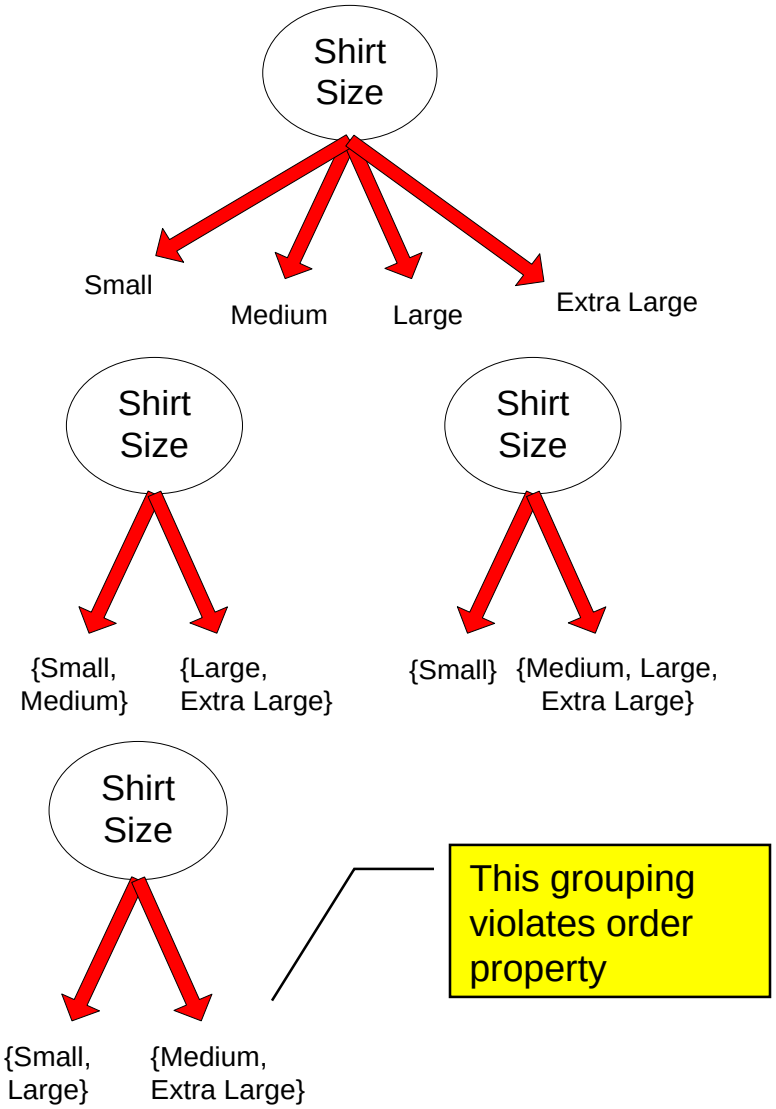


- **Binary split**
  - **Divides values into two subsets**

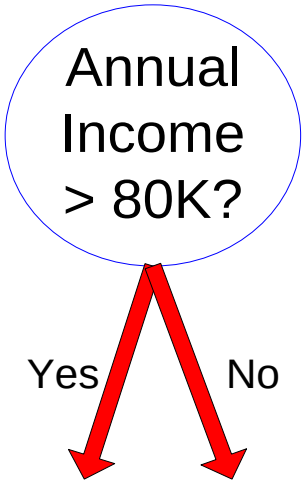


Test Condition for Ordinal Attributes

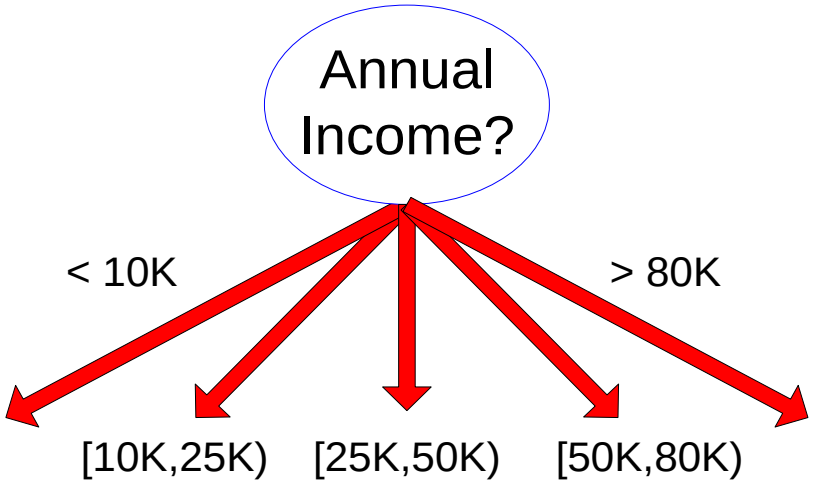
- **Multi-way split**
  - Use as many partitions as distinct values
- **Binary split**
  - Divides values into two subsets
  - Preserve order property among attribute values



Test Condition for Continuous Attributes



(i) Binary split



(ii) Multi-way split

上海交通大学RFID与物联网研究所

Splitting Based on Continuous Attributes

- Different ways of handling
  - —Discretization to form an ordinal categorical attribute

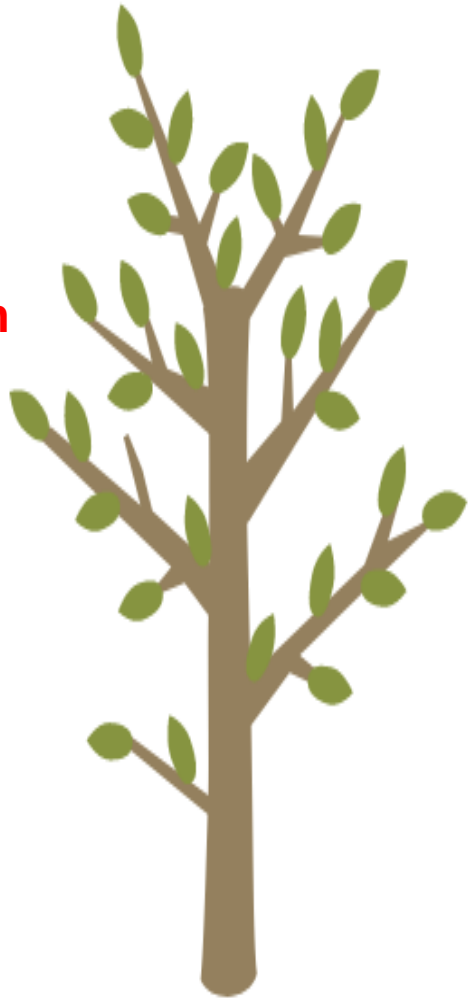
    Ranges can be found by equal interval bucketing, equal frequency bucketing (percentiles), or clustering
    - Static – discretize once at the beginning
    - Dynamic – repeat at each node

  - —Binary Decision: (A < v) or (A ≥ v)
    - consider all possible splits and finds the best cut
    - can be more compute intensive

上海交通大学RFID与物联网研究所

Decision Tree Classifier

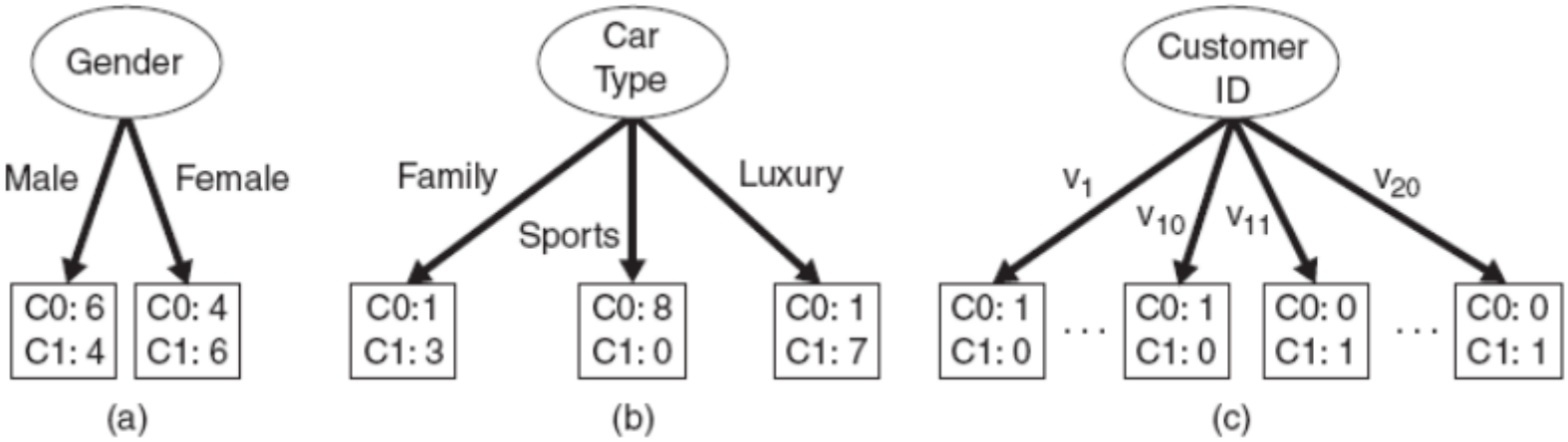
- Overview
- Basic Algorithm to Build a Decision Tree
- Methods for Expressing Attribute Test Conditions
- Measures for Selecting an Attribute Test Condition
- Algorithm for Decision Tree Induction
- Classic decision tree algorithm
- Example Application: Web Robot Detection
- Characteristics of Decision Tree Classifiers



How to determine the Best Split

– **Before Splitting: 10 records of class 0, 10 records of class 1**

C0: 10
C1: 10



Which test condition is the best?

How to determine the Best Split

- Greedy approach
  - Nodes with homogeneous class distribution are preferred

- Need a measure of node impurity

C0:	**5**
C1:	**5**

Non-homogeneous,

High degree of impurity

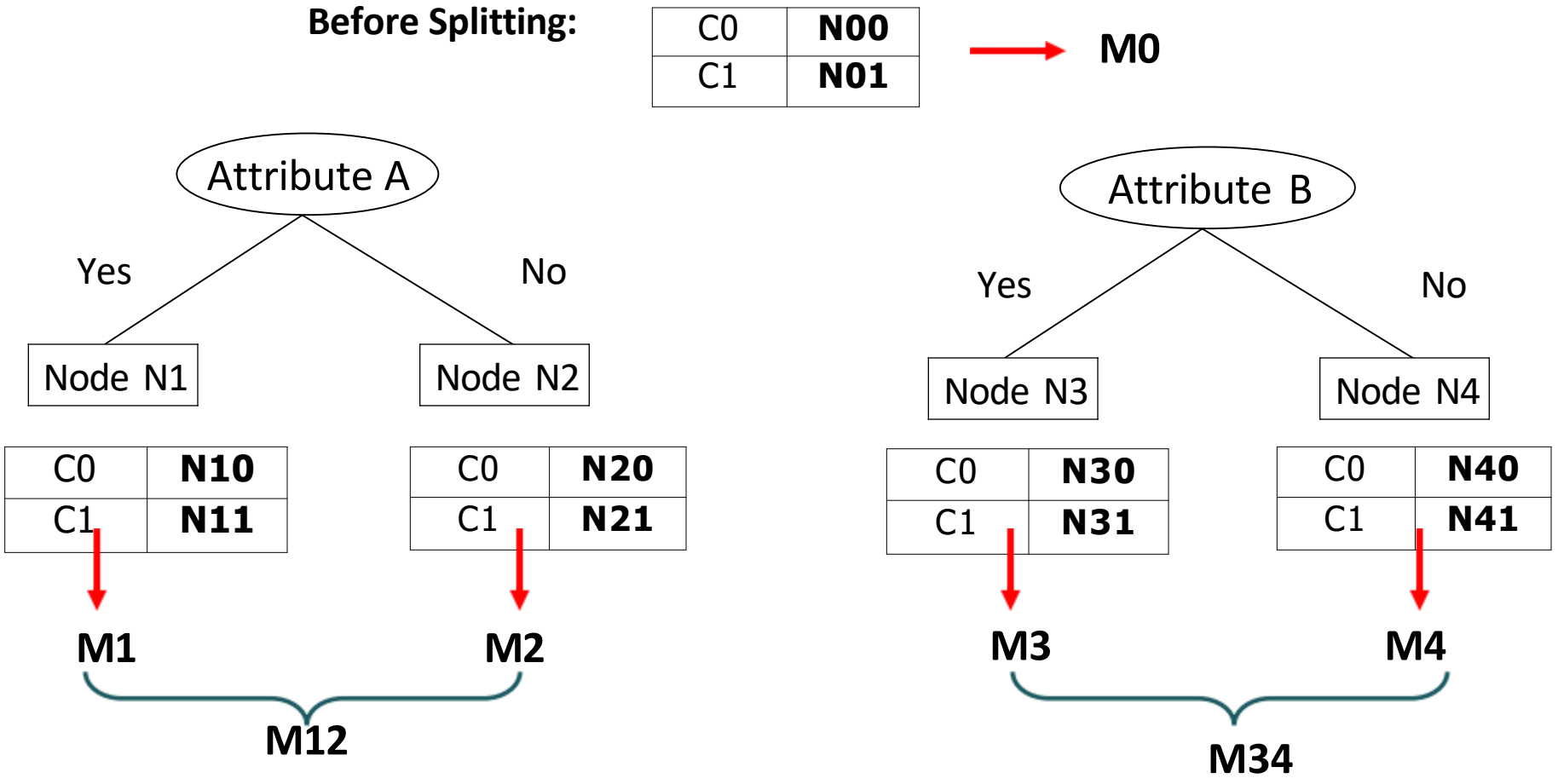
C0:	**9**
C1:	**1**

Homogeneous,

Low degree of impurity

Find the Best Split -General Framework

– Assume we have a measure M that tells us how "pure" a node is.



Gain = M0 – M12 vs M0 – M34 → Choose best split

Find the Best Split -General Framework

- Measures of Node Impurity 单结点的不纯性度量
- Collective Impurity of Child Nodes 子结点的集合不纯性
- Identifying the best attribute test condition 确定最佳属性测试条件
- Splitting of Qualitative Attributes 定性属性的划分
- Binary Splitting of Qualitative Attributes 定性属性的二元划分
- Binary Splitting of Quantitative Attribut定量属性的二元划分
- Gain Ratio 增益率



Find the Best Split -General Framework

- Measures of Node Impurity 单结点的不纯性度量
  - —Gini Index
  - —Entropy
  - —Classification error
- Collective Impurity of Child Nodes 子结点的集合不纯性
- Identifying the best attribute test condition 确定最佳属性测试条件
- Splitting of Qualitative Attributes 定性属性的划分
- Binary Splitting of Qualitative Attributes 定性属性的二元划分
- Binary Splitting of Quantitative Attribut定量属性的二元划分
- Gain Ratio 增益率



Gini Index 属性选择度量 之 基尼系数

- 设t是分类回归树中的某个节点，k为当前属性下测试输出的类别树，p(j|t)为节点t中样本测试输出取类别j的概率

$$\mathrm{Gini}(t)=1-\sum_{j=1}^{k} p^2(j|t)$$

- 对节点t，G(t)越小，意味着该节点中所包含的样本越集中在某一类上，即该节点越纯，否则越不纯，差异性就越大
  - 当节点样本的测试输出均取同一类取值时，输出变量取值的差异性最小，Gini系数为0
  - 当各类别取概率值相等时，测试输出取值的差异性最大， Gini系数也最大，为1-1/k，其中k为目标变量的类别数

Examples for computing GINI

$$GINI(t) = 1 - \sum_j p(j \mid t)^2$$

C1	0
C2	6

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Gini = 1 – $P(C1)^2$ – $P(C2)^2$ = 1 – 0 – 1 = **0**

C1	1
C2	5

P(C1) = 1/6 P(C2) = 5/6

Gini = 1 – $(1/6)^2$ – $(5/6)^2$ = **0.278**

C1	2
C2	4

P(C1) = 2/6 P(C2) = 4/6

Gini = 1 – $(2/6)^2$ – $(4/6)^2$ = **0.444**

C1	0
C2	6
Gini=0.000	

C1	1
C2	5
Gini=0.278	

C1	2
C2	4
Gini=0.444	

C1	3
C2	3
Gini=0.500	

Maximal impurity here is ½ = .5

Find the Best Split -General Framework

- Measures of Node Impurity 单结点的不纯性度量
  - —Gini Index
  - —Entropy
  - —Classification error
- Collective Impurity of Child Nodes 子结点的集合不纯性
- Identifying the best attribute test condition 确定最佳属性测试条件
- Splitting of Qualitative Attributes 定性属性的划分
- Binary Splitting of Qualitative Attributes 定性属性的二元划分
- Binary Splitting of Quantitative Attribut定量属性的二元划分
- Gain Ratio 增益率



Information Gain 属性选择度量 之 信息增益

- 信息增益
  - 在树的每个节点上使用信息增益度量选择测试属性
  - 根据当前节点对应的训练样本，计算各属性的信息增益，然后选择具有最高信息增益的属性作为分裂属性来做样本划分
- 信息增益计算方式
  - 信息熵（entropy，ɛntrəpi ，熵，平均信息量，负熵 ）：用于度量一个属性的信息量
    - ✓ 假定S为训练集，S的目标属性C具有m个可能的类标号值，$C=\{C_1,C_2,\ldots,C_m\}$，假定训练集S中C_i在所有样本中出现的概率为$p_i(i=1,2,3,\ldots,m)$，则该训练集S所包含的信息熵定义为

$$\text{Entropy}(S)=\text{Entropy}(p_1,p_2,\cdots,p_m)=-\sum_{i=1}^{m}p_i\log_2(p_i)$$

    - ✓ 熵越小，表示样本对目标属性的分布越纯。熵越大，表示样本对目标属性的分布越乱
      - 熵为0则意味着所有样本的目标属性取值相同
      - 当S中不同类别的记录数相当时，取得最大值$\log 2^m$,m为类别的个数，对于两个类别来说，最大值为1

Examples for computing Entropy

$$\text{Entropy}(S) = \text{Entropy}(p_1, p_2, \cdots, p_m) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Entropy = – 0 $\log_2 0$ – 1 $\log_2 1$ = – 0 – 0 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Entropy = – (1/6) $\log_2$ (1/6) – (5/6) $\log_2$ (1/6) = 0.65

C1	**3**
C2	**3**

P(C1) = 3/6 P(C2) = 3/6

Entropy = – (3/6) $\log_2$ (3/6) – (3/6) $\log_2$ (3/6) = 1

Find the Best Split -General Framework

- Measures of Node Impurity 单结点的不纯性度量
  - —Gini Index
  - —Entropy
  - —Classification error
- Collective Impurity of Child Nodes 子结点的集合不纯性
- Identifying the best attribute test condition 确定最佳属性测试条件
- Splitting of Qualitative Attributes 定性属性的划分
- Binary Splitting of Qualitative Attributes 定性属性的二元划分
- Binary Splitting of Quantitative Attribut定量属性的二元划分
- Gain Ratio 增益率



Classification error 属性选择度量之分类误差

- 设t是分类回归树中的某个节点，k为当前属性下测试输出的类别树，p(i|t)为节点t中样本测试输出取类别j的概率

$$\text{Classification error} = 1 - \max_i [p_i(t)]$$

- 对节点t，CE越小，意味着该节点中所包含的样本越集中在某一类上，即该节点越纯，否则越不纯，差异性就越大
  - 当节点样本的测试输出均取同一类取值时，输出变量取值的差异性最小，CE为0
  - 当各类别取概率值相等时，测试输出取值的差异性最大， CE也最大，为1-1/k，其中k为目标变量的类别数

Examples for computing Classification error

$$\text{Error}(t) = 1 - \max_i p(i \mid t)$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Error = 1 – max (0, 1) = 1 – 1 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Error = 1 – max (1/6, 5/6) = 1 – 5/6 = 1/6

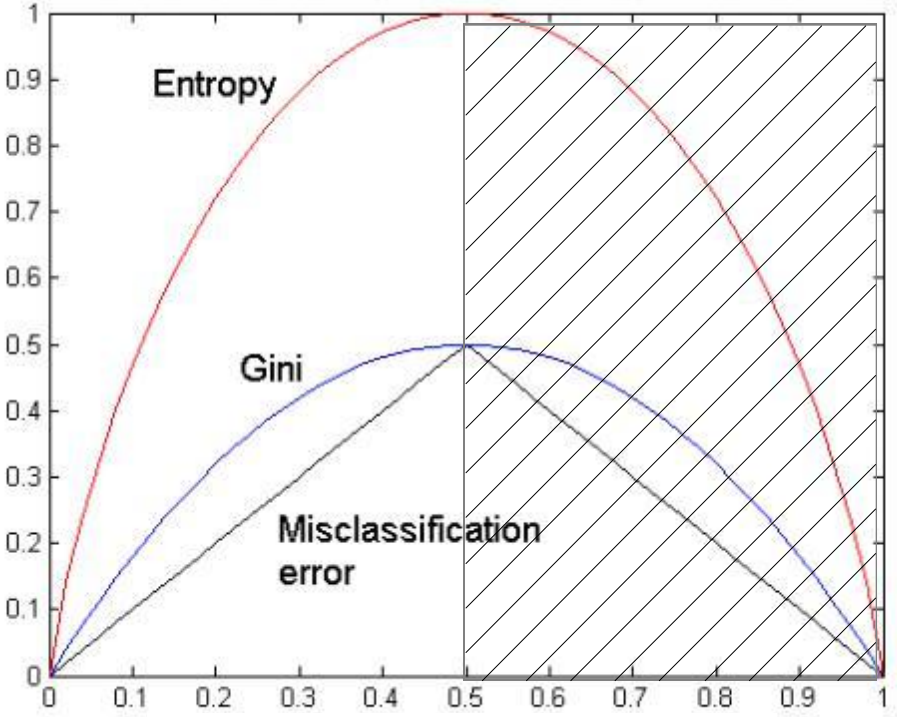
C1	**3**
C2	**3**

P(C1) = 3/6 P(C2) = 3/6

Error = 1 – max (3/6, 3/6) = 1 – 3/6 = .5

Comparison among Splitting Criteria

- 展示了不纯性度量的一致性，即如果结点 N_1 的熵比结点 N_2 低，那么 N_1 的基尼指数 (Gini index)和误差率也将低于N_2 。 尽管它们一致，但属性选择仍然因不纯性度量而不同



For a 2-class problem: Probability of the majority class p is always > .5

Note: The order is the same no matter what splitting criterion is used, however, the gain (differences) are not.

Find the Best Split -General Framework

- Measures of Node Impurity 单结点的不纯性度量
- Collective Impurity of Child Nodes 子结点的集合不纯性
- Identifying the best attribute test condition 确定最佳属性测试条件
- Splitting of Qualitative Attributes 定性属性的划分
- Binary Splitting of Qualitative Attributes 定性属性的二元划分
- Binary Splitting of Quantitative Attribut定量属性的二元划分
- Gain Ratio 增益率

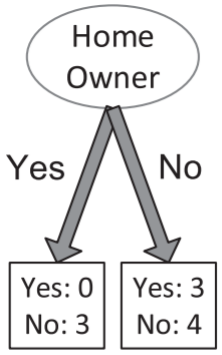


子结点的集合不纯性

- 设一个属性测试条件，该条件将包含 N 个训练实例的结点拆分为k 个子结点{v_1, v_2,…,v_k}， 其中每个子结点表示由属性测试的 k 个结果之一产生的数据划分条件。设N(v_j) 为与不纯性为I(v_j) 的子结点 v_j相关联的训练实例的数量。由于父结点中的训练实例在N(v_j)/N 次数内到达结点v_j ，因此可以通过对子结点的不纯性度量进行加权求和来计算子结点的集合不纯性

$$I(\text{children}) = \sum_{j=1}^{k} \frac{N(v_j)}{N} I(v_j)$$

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



(a)

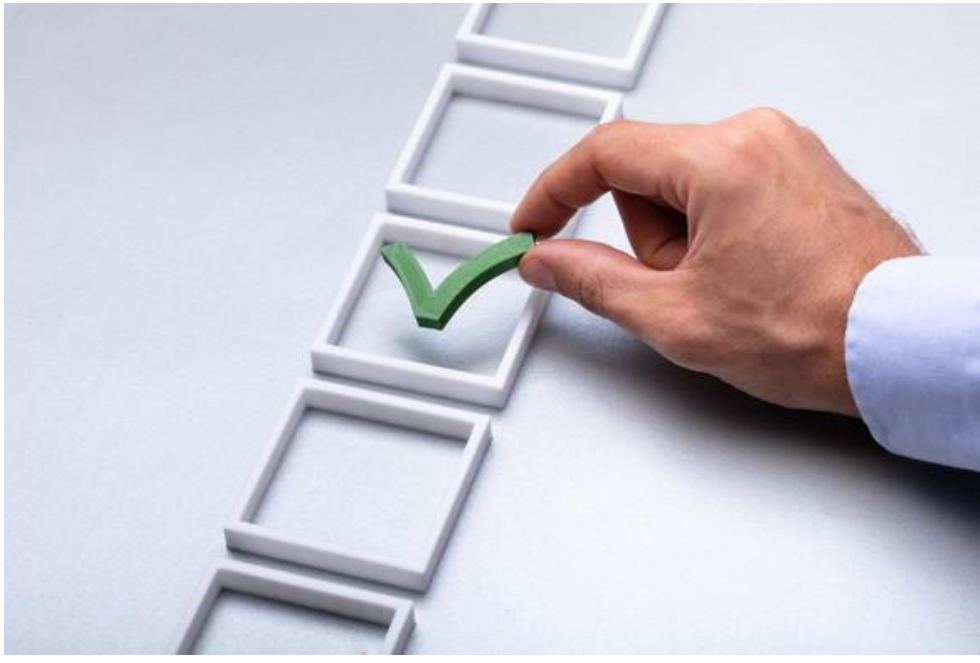
$$I(\texttt{Home Owner = yes}) = -\frac{0}{3}\log_2\frac{0}{3} - \frac{3}{3}\log_2\frac{3}{3} = 0$$

$$I(\texttt{Home Owner = no}) = -\frac{3}{7}\log_2\frac{3}{7} - \frac{4}{7}\log_2\frac{4}{7} = 0.985$$

$$I(\texttt{Home Owner}) = \frac{3}{10} \times 0 + \frac{7}{10} \times 0.985 = 0.690$$

Find the Best Split -General Framework

- Measures of Node Impurity 单结点的不纯性度量
- Collective Impurity of Child Nodes 子结点的集合不纯性
- Identifying the best attribute test condition 确定最佳属性测试条件
- Splitting of Qualitative Attributes 定性属性的划分
- Binary Splitting of Qualitative Attributes 定性属性的二元划分
- Binary Splitting of Quantitative Attribut定量属性的二元划分
- Gain Ratio 增益率



确定最佳属性测试条件

- 为了确定属性测试条件的优劣，需要比较父结点(在划分之前)的不纯性和子结点的不纯性的加权程度(分裂之后)，差异越大，测试条件越好。这种差异(△)也称为属性测试条件的纯度增益(gain), 定义如下：

$$\triangle = I(\text{父结点}) - I(\text{子结点})$$

  —I(父结点)是划分前结点的不纯性

  —I(子结点)是划分后结点的加权不纯性度量

- 可以证明，由于I(父结点)≥I(子结点)，对于上述任何合理的度量，增益是非负的。增益越高，子结点相对于父结点的类越纯洁。决策树学习算法中的分裂准则选择最大增益的属性测试条件
- 在给定结点处最大化增益相当于最小化其子项的加权不纯性度量，因为对于所有候选属性测试条件，I(父结点)是相同的
- 当熵用作不纯性度量时，熵的差异通常称为信息增益(information gain)——$\triangle_{info}$

Information Gain 属性选择度量 之 信息增益

- 信息增益计算方式
- 信息增益
  - ✓ 用属性A划分样本数据集S，划分前样本数据集的不纯程度（熵）和划分后样本数据集的不纯程度（熵）的差值

$$\text{Gain}(S, A) = \text{Entropy}(S) - \text{Entropy}_A(S)$$

  - ✓ 按属性A划分S后的样本子集的熵定义：假定属性A具有K个不同的取值，将S划分为k个样本子集{S_1，S_2，…，S_k}，则按属性A划分S后的样本子集的信息熵如下，其中$|S_i|$（i=1,2,…,k）为样本子集S_i中包含的样本数，$|S|$为样本集S中包含的样本数

$$\text{Entropy}_A(S) = \sum_{i=1}^{k} \frac{|S_i|}{|S|} \text{Entropy}(S_i)$$

  - ✓ 信息增益越大，说明使用属性A划分后的样本子集越纯，越有利于分类

Find the Best Split -General Framework

- Measures of Node Impurity 单结点的不纯性度量
- Collective Impurity of Child Nodes 子结点的集合不纯性
- Identifying the best attribute test condition 确定最佳属性测试条件
- Splitting of Qualitative Attributes 定性属性的划分
- Binary Splitting of Qualitative Attributes 定性属性的二元划分
- Binary Splitting of Quantitative Attribut定量属性的二元划分
- Gain Ratio 增益率



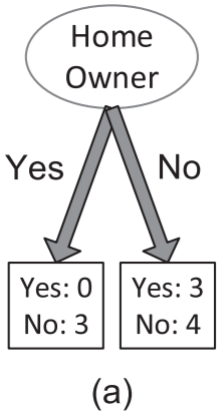
定性属性的划分

- 定性属性为有房者和婚姻状况。在父结点处的初始类分布为(0.3,0.7),由于训练数据中有3个实例为是，7个实例为否，因此有：

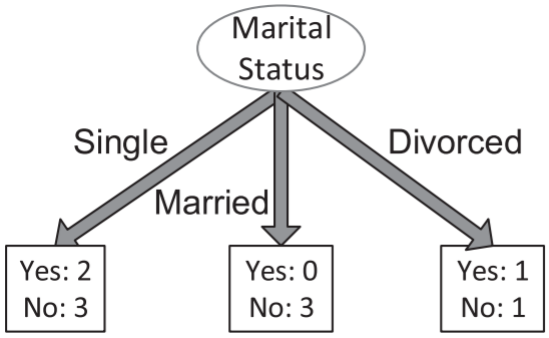
$$I(\text{父结点}) = -\frac{3}{10}\log_2\frac{3}{10} - \frac{7}{10}\log_2\frac{7}{10} = 0.881$$

- 有房者和婚姻状况的信息增益分别为：
  - $\triangle_{info}$(有房者)=0.881-0.690 =0.191
  - $\triangle_{info}$(婚姻状况)=0.881-0.680 =0.195
- 由于婚姻状况的加权熵较低，因此其信息增益较高，将被用于划分

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



(a)



Find the Best Split -General Framework

- Measures of Node Impurity 单结点的不纯性度量
- Collective Impurity of Child Nodes 子结点的集合不纯性
- Identifying the best attribute test condition 确定最佳属性测试条件
- Splitting of Qualitative Attributes 定性属性的划分
- Binary Splitting of Qualitative Attributes 定性属性的二元划分
- Binary Splitting of Quantitative Attribut定量属性的二元划分
- Gain Ratio 增益率



定性属性的二元划分

- 考虑仅使用二元划分和基尼指数作为不纯性度量来构建决策树
- 属性 有房者和婚姻状况的4个候选划分标准的例子。由于训练集中有3名借款人违约，另有7 名借款人偿还了贷款,因此划分前的父结点的基尼指数为

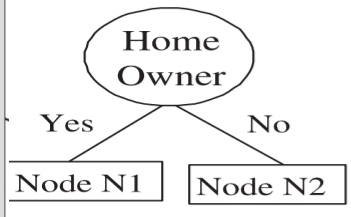
$$1-\left(\frac{3}{10}\right)^2-\left(\frac{7}{10}\right)^2=0.420$$

$$\left(\frac{3}{10}\right)\times 0+\left(\frac{7}{10}\right)\times 0.490=0.343$$

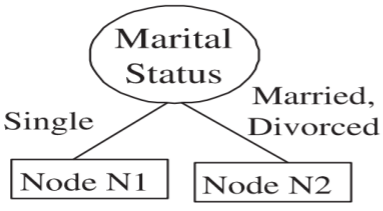
- 如果选择有房者作为划分属性，则子结点N_1 和N_2 的基尼指数分别为0和0.490。这些子结点的加权平均基尼指数是
- 此处的权重代表分配给每个子结点训练实例的比例。使用有房者作为划分属性的增益为 0.420-0.343=0.077

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

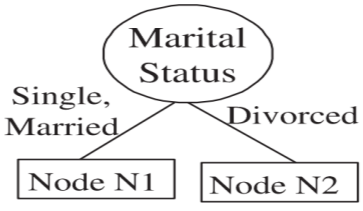
	Parent
No	7
Yes	3
Gini = 0.420	



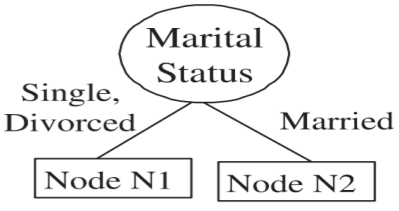
	N1	N2
No	3	4
Yes	0	3
Gini= 0.343		



	N1	N2
No	3	4
Yes	2	1
Gini= 0.400		



	N1	N2
No	6	1
Yes	2	1
Gini= 0.400		



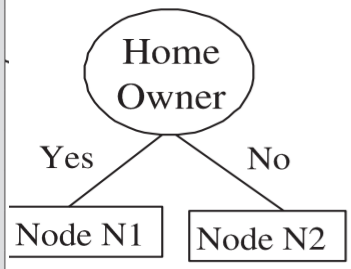
	N1	N2
No	4	3
Yes	3	0
Gini= 0.343		

定性属性的二元划分

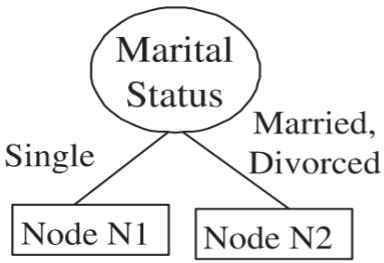
- 对属性婚姻状况应用二元划分。由于婚姻状况是一个具有三种结果的标称属性，因此有三种可能的方式对属性值进行二元划分。 图中显示了每个候选二元划分的子结点的加权平均基尼指数
- 根据计算结果，有房者和使用婚姻状况的二元分组显然是最好的候选，因为它们都产生最低的加权平均基尼指数。如果属性值的二元划分不违反顺序属性，则二元划分也可用于序数属性

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

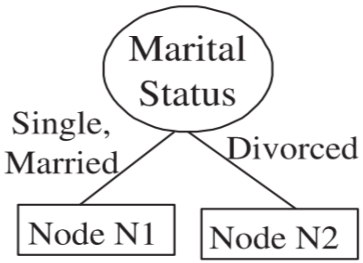
	Parent
No	7
Yes	3
Gini = 0.420	



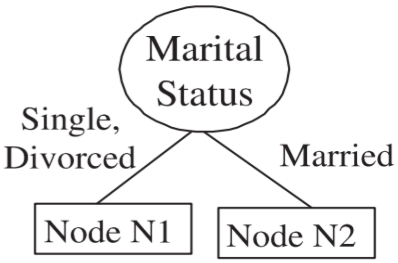
	N1	N2
No	3	4
Yes	0	3
Gini= 0.343		



	N1	N2
No	3	4
Yes	2	1
Gini= 0.400		



	N1	N2
No	6	1
Yes	2	1
Gini= 0.400		



	N1	N2
No	4	3
Yes	3	0
Gini= 0.343		

Find the Best Split -General Framework

- Measures of Node Impurity 单结点的不纯性度量
- Collective Impurity of Child Nodes 子结点的集合不纯性
- Identifying the best attribute test condition 确定最佳属性测试条件
- Splitting of Qualitative Attributes 定性属性的划分
- Binary Splitting of Qualitative Attributes 定性属性的二元划分
- Binary Splitting of Quantitative Attribut定量属性的二元划分
- Gain Ratio 增益率



定量属性的二元划分

- 考虑为上述拖欠贷款分类问题确定最佳二元划分“年收入≤r”的问题。尽管r 可以在训练集中年收入的最小值和最大值之间取任意值，但只需将训练集中观察到的年收入值视为候选划分位置就足够了。对于每个候选r，训练集被扫描一次以计算年收入小于或大于r的借款人数量及其类别比例
- 然后，计算每个候选划分位置的基尼指数，并选择产生r 的最小值。在每个候选划分位置计算基尼指数需要 O(N) 次操作，其中N 是训练实例的数量。由于最多有N个候选，所以暴力法的计算复 杂度为 $O(N^2)$

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

定量属性的二元划分

- 通过下述方法可以将此问题的计算复杂度降至O(NlogN)
  - 首先根据年收入将训练实例排序，所需时间为O(NlogN)。从两个相邻的排过序的属性值中选择中间值作为候选划分点，得到候选划分点为55000美元、65000美元、72500美元等
  - 对于第一位候选，由于无年收入低于或等于55000美元，年收入<55000美元的子结点的基尼指数等于零。相比之下，年收入大于 55000美元的结点有3个实例是是，7个是否，该结点的基尼指数是0.420。第一候选划分位置r=55000 美元的加权平均基尼指数等于
    0×0+1×0.420=0.420
  - 对于第二个候选r=65000， 通过简单更新上一个候选的类分布，就可得到该候选类分布。这是因为，当r从55000美元增加到65000美元时，只有一个受此变化影响的训练实例。通过检查受影响的训练实例的类别标签，可以获得新的类分布。例如，当r增加到65000美元时，训练集中只有一个借款人，年收入为60000美元，受此变化影响。由于借款人的类别标签为否，所以类别否的计数从0增加到1(年收入≤65000美元)，并从7减少到6(年收入>65000 美元)，如图所示。是类的分布保持不变。新的候选划分的基尼指数为0.400

Find the Best Split -General Framework

- Measures of Node Impurity 单结点的不纯性度量
- Collective Impurity of Child Nodes 子结点的集合不纯性
- Identifying the best attribute test condition 确定最佳属性测试条件
- Splitting of Qualitative Attributes 定性属性的划分
- Binary Splitting of Qualitative Attributes 定性属性的二元划分
- Binary Splitting of Quantitative Attribut定量属性的二元划分
- Gain Ratio 增益率

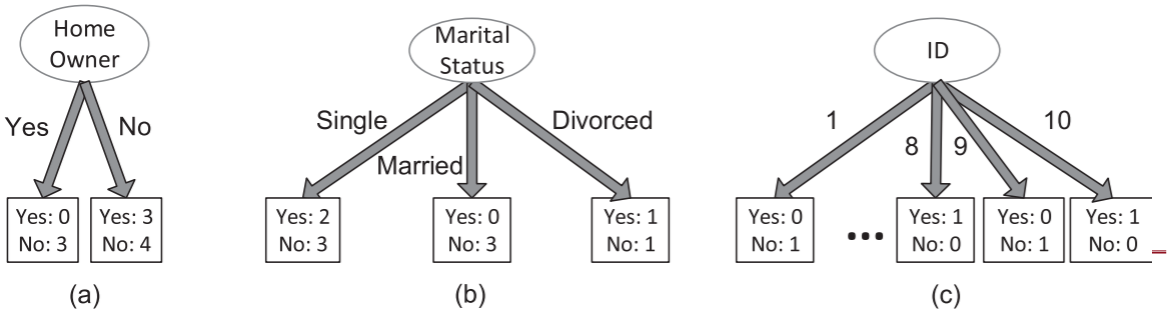


Gain Ratio 增益率

- 熵和基尼指数等不纯性度量存在一个潜在的局限，即更容易选择具有大量不同值的定性属性。图给出了表列出的划分数据集的三个候选属性。选择属性婚姻状况优于选择属性有房者，因为前者提供了更大的信息增益。但是，如果将它们与顾客 **ID** 进行比较，后者因为加权熵和基尼指数等于零，会生成信息增益更大的最纯划分。然而，顾客**ID** 并不是一个很好的划分属性。这个例子表明单一的低不纯性结点不足以找到良好的属性测试条件
- 子结点越多，决策树越复杂，更容易出现过拟合。因此，在决定最佳属性测试条件时，还应该考虑划分属性产生的子结点数量。有两种方法可以解决这个问题
  - 一种方法是仅生成二元决策树，从而避免使用不同数量的划分来处理属性。决策树分类器(如 **CART**) 就是使用此策略
  - 另一种方法是修改划分标准以考虑属性生成的划分数量。例如，在 **C4.5** 决策树算法中，使用称为增益率**(gain ratio)**的度量来补偿产生大量子结点的属性

D	Home Owner	Marital Status	Annual Income	Defaulted Borrower
	Yes	Single	125K	No
	No	Married	100K	No
	No	Single	70K	No
	Yes	Married	120K	No
	No	Divorced	95K	Yes
	No	Married	60K	No
	Yes	Divorced	220K	No
	No	Single	85K	Yes
	No	Married	75K	No
0	No	Single	90K	Yes

$$\text{Gain ratio} = \frac{\Delta_{\text{info}}}{\text{Split Info}} = \frac{\text{Entropy(Parent)} - \sum_{i=1}^{k} \frac{N(v_i)}{N}\text{Entropy}(v_i)}{-\sum_{i=1}^{k} \frac{N(v_i)}{N}\log_2 \frac{N(v_i)}{N}}$$



Gain Ratio 增益率

Customer ID	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

$$\text{Entropy(parent)} = -\frac{10}{20}\log_2\frac{10}{20} - \frac{10}{20}\log_2\frac{10}{20} = 1$$

If `Gender` is used as attribute test condition:

$$\text{Entropy(children)} = \frac{10}{20}\left[-\frac{6}{10}\log_2\frac{6}{10} - \frac{4}{10}\log_2\frac{4}{10}\right] \times 2 = 0.971$$

$$\text{Gain Ratio} = \frac{1-0.971}{-\frac{10}{20}\log_2\frac{10}{20} - \frac{10}{20}\log_2\frac{10}{20}} = \frac{0.029}{1} = 0.029$$

If `Car Type` is used as attribute test condition:

$$\text{Entropy(children)} = \frac{4}{20}\left[-\frac{1}{4}\log_2\frac{1}{4} - \frac{3}{4}\log_2\frac{3}{4}\right] + \frac{8}{20}\times 0 + \frac{8}{20}\left[-\frac{1}{8}\log_2\frac{1}{8} - \frac{7}{8}\log_2\frac{7}{8}\right] = 0.380$$

$$\text{Gain Ratio} = \frac{1-0.380}{-\frac{4}{20}\log_2\frac{4}{20} - \frac{8}{20}\log_2\frac{8}{20} - \frac{8}{20}\log_2\frac{8}{20}} = \frac{0.620}{1.52} = 0.41$$

Finally, if `Customer ID` is used as attribute test condition:

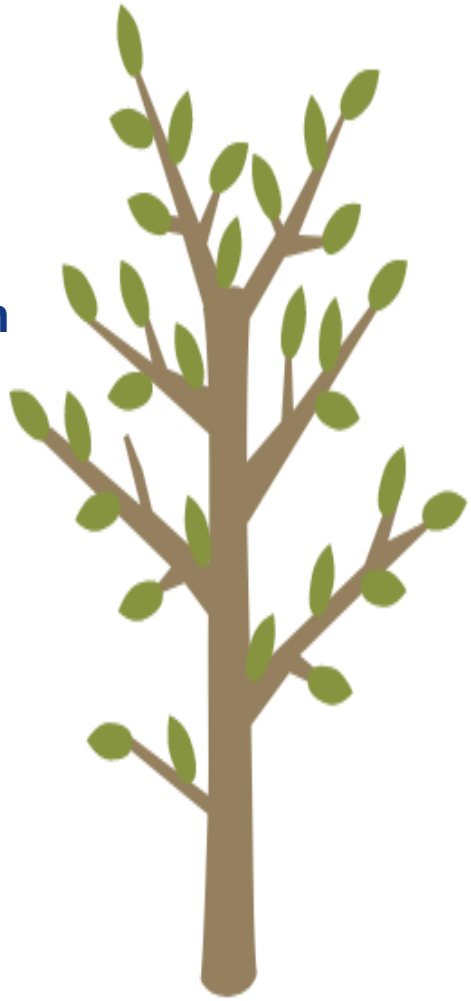
$$\text{Entropy(children)} = \frac{1}{20}\left[-\frac{1}{1}\log_2\frac{1}{1} - \frac{0}{1}\log_2\frac{0}{1}\right] \times 20 = 0$$

$$\text{Gain Ratio} = \frac{1-0}{-\frac{1}{20}\log_2\frac{1}{20}\times 20} = \frac{1}{4.32} = 0.23$$

尽管顾客ID 具有最高的信息增益，但其增益率低于车型，因为前者会产生更 大数量的划分

Decision Tree Classifier

- Overview
- Basic Algorithm to Build a Decision Tree
- Methods for Expressing Attribute Test Conditions
- Measures for Selecting an Attribute Test Condition
- Algorithm for Decision Tree Induction
- Classic decision tree algorithm
- Example Application: Web Robot Detection
- Characteristics of Decision Tree Classifiers



由训练元组归纳决策树的基本算法

Algorithm: Generate_decision_tree. Generate a decision tree from the training tuples of data partition, *D*.

Input:

- Data partition, *D*, which is a set of training tuples and their associated class labels;
- *attribute_list*, the set of candidate attributes;
- *Attribute_selection_method*, a procedure to determine the splitting criterion that "best" partitions the data tuples into individual classes. This criterion consists of a *splitting_attribute* and, possibly, either a *split-point* or *splitting subset*.

Output: A decision tree.

Method:

(1) create a node *N*;
(2) **if** tuples in *D* are all of the same class, *C*, **then**
(3) return *N* as a leaf node labeled with the class *C*;
(4) **if** *attribute_list* is empty **then**
(5) return *N* as a leaf node labeled with the majority class in *D*; // majority voting
(6) apply **Attribute_selection_method**(*D*, *attribute_list*) to **find** the "best" *splitting_criterion*;
(7) label node *N* with *splitting_criterion*;
(8) **if** *splitting_attribute* is discrete-valued **and**
multiway splits allowed **then** // not restricted to binary trees
(9) *attribute_list* ← *attribute_list* − *splitting_attribute*; // remove *splitting_attribute*
(10) **for each** outcome *j* of *splitting_criterion*
// partition the tuples and grow subtrees for each partition
(11) let D_j be the set of data tuples in *D* satisfying outcome *j*; // a partition
(12) **if** D_j is empty **then**
(13) attach a leaf labeled with the majority class in *D* to node *N*;
(14) **else** attach the node returned by **Generate_decision_tree**(D_j, *attribute_list*) to node *N*;
endfor
(15) return *N*;

各类决策树算法之间的差别

- 基本算法对于树的每一层，需要扫描一遍D 中的元组。在处理大型数据库时，这可能导致很长的训练时间和内存不足

基本算法的计算复杂度

给定训练集 D，算法的计算复杂度为 $O(n \times |D| \times \log(|D|))$，其中 n 是描述 D 中元组的属性个数，$|D|$ 是 D 中的训练元组数。这意味以 $|D|$ 个元组产生一棵树的计算开销最多为 $n \times |D| \times \log(|D|)$。

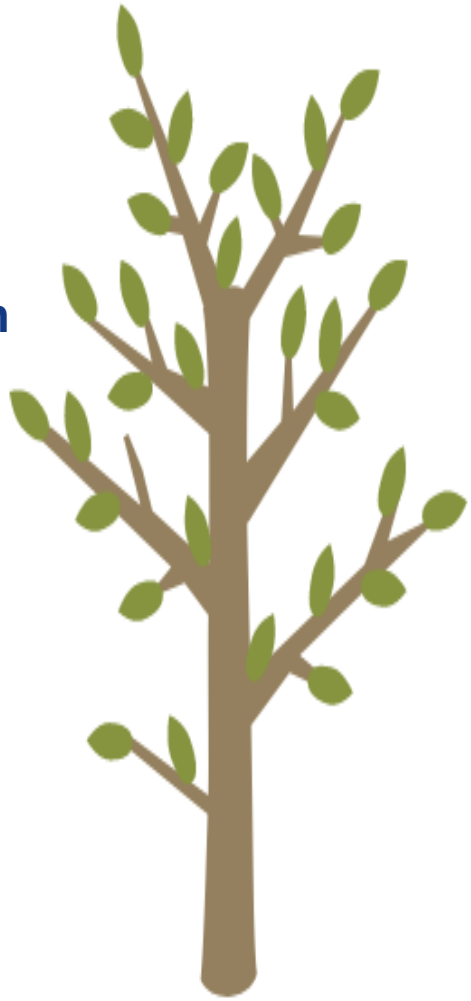
- 关于决策树归纳的改进算法很多，主要差别体现在

  —在创建树时如何选择属性

  —用于剪枝的机制

Decision Tree Classifier

- Overview
- Basic Algorithm to Build a Decision Tree
- Methods for Expressing Attribute Test Conditions
- Measures for Selecting an Attribute Test Condition
- Algorithm for Decision Tree Induction
- Classic decision tree algorithm
- Example Application: Web Robot Detection
- Characteristics of Decision Tree Classifiers



决策树归纳算法

- 在20世纪70年代后期和2 0世纪80年代初期，机器学习研究人员J.Ross Quinlan开发了决策树算法，称为迭代的二分器（Iterative Dichotomister，ID3）。这项工作扩展了E.B. Hunt，J.Marin和P.T.Stone的概念学习系统。
- Quinlan后来提出了C4.5（ID3的后继），成为了新的监督学习算法的性能比较基准
- 1984年，多位统计学家（L.Breiman、J.Friedman、R.Olshen、C.Stone）出版著作《Classification and Regression Trees》（CART），介绍二叉决策树的产生
- ID3和CART大约同时独立地发明，但是从训练元组学习决策树都采用了类似的方法，这两个基础算法引发了决策树归纳研究的旋风

在创建树时如何选择属性
Measures of Impurity 不纯性度量



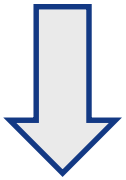
Gini Index



CART



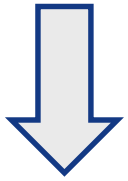
Entropy



ID3



Gain Ratio



C4.5

CART算法简介

- CART（Classification and Regression Tree，分类与回归树）算法由斯坦福大学和加州大学伯克利分校的Breiman等人与1984年提出
- CART决策树能够处理连续属性数据和标称属性作为预测变量和目标变量下的分类
  - —当输出变量是标称属性数据时，所建立的决策树被称为分类树（classfication tree），用于分类的预测
  - —当输出变量是数值型的数据时，所建立的决策树被称为回归树（regression tree），用于数值的预测
- 分类回归树算法同样决策树增长和决策树剪枝两个过程，算法采用二元递归划分方法，将当前的样本集分为两个样本子集，使得生成的决策树的每个非叶节点都有两个分支，CART算法生成的决策树是结构简洁的二叉树

Gini Index 属性选择度量 之 基尼系数

- 设t是分类回归树中的某个节点，k为当前属性下测试输出的类别树，p(j|t)为节点t中样本测试输出取类别j的概率

$$\mathrm{Gini}(t)=1-\sum_{j=1}^{k} p^2(j|t)$$

- 对节点t，G(t)越小，意味着该节点中所包含的样本越集中在某一类上，即该节点越纯，否则越不纯，差异性就越大
  - 当节点样本的测试输出均取同一类取值时，输出变量取值的差异性最小，Gini系数为0
  - 当各类别取概率值相等时，测试输出取值的差异性最大， Gini系数也最大，为1-1/k，其中k为目标变量的类别数

Gini Index 属性选择度量 之 基尼系数

- 设 t是一个节点， ξ 为该节点的一个属性分支条件，该分支条件将节点t中样本分别分到左分支S_L和右分支S_R右，则称下式为在**分支条件 ξ 下节点t的差异性损失**，其中，gini(t)为划分前测试输出的Gini系数，$|S_R|$和$|S_L|$分别表示划分后左右分支的样本个数

$$\Delta gini(\xi,t)=gini(t)-\frac{|S_R|}{|S_L|+|S_R|}gini(t_R)-\frac{|S_L|}{|S_L|+|S_R|}gini(t_L)$$

- 为了使节点t尽可能纯，需要选择某个属性分支条件 ξ，使该节点的差异性损失尽可能大，用 ξ(t)表示所考虑的分支条件 ξ 的全体，则选择分支条件应为

$$\xi_{\max}=\arg\max_{\xi\in\xi(t)}\Delta gini(\xi,t)$$

arg作为变量（即自变量）的英文缩写

CART算法构造决策树：一个例子

序号	是否有房	婚姻状况	年收入	拖欠货款
1	yes	single	125k	no
2	no	married	100k	no
3	no	single	70k	no
4	yes	married	120k	no
5	no	divorced	95k	yes
6	no	married	60k	no
7	yes	divorced	220k	no
8	no	single	85k	yes
9	no	married	75k	no
10	no	single	90k	yes

在创建树时如何选择属性
Measures of Impurity 不纯性度量





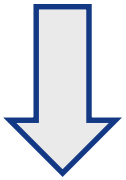


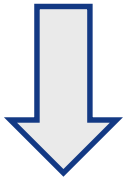
Gini Index

Entropy

Gain Ratio







CART

ID3

C4.5

决策树归纳经典算法
ID3基本原理

- 由Quinlan在1986年提出
- 使用信息增益作为属性选择标准
- 算法原理
  - 首先检测所有属性，选择信息增益值最大属性产生决策树节点
  - 由该属性的不同取值建立分支
  - 再对各分支的子集递归调用该方法建立决策树节点的分支
  - 直到所有子集仅包含同一个类别的数据为止，最后得到一棵决策树，对所有的新样本进行分类

Information Gain 属性选择度量 之 信息增益

- 信息增益
  - 在树的每个节点上使用信息增益度量选择测试属性
  - 根据当前节点对应的训练样本，计算各属性的信息增益，然后选择具有最高信息增益的属性作为分裂属性来做样本划分
- 信息增益计算方式
  - 信息熵（entropy，ɛntrəpi ，熵，平均信息量，负熵 ）：用于度量一个属性的信息量
    - ✓ 假定S为训练集，S的目标属性C具有m个可能的类标号值，$C=\{C_1,C_2,\ldots,C_m\}$，假定训练集S中C_i在所有样本中出现的概率为$p_i(i=1,2,3,\ldots,m)$，则该训练集S所包含的信息熵定义为

$$\text{Entropy}(S)=\text{Entropy}(p_1,p_2,\cdots,p_m)=-\sum_{i=1}^{m}p_i\log_2(p_i)$$

    - ✓ 熵越小，表示样本对目标属性的分布越纯。熵越大，表示样本对目标属性的分布越乱
      - 熵为0则意味着所有样本的目标属性取值相同
      - 当S中不同类别的记录数相当时，取得最大值$\log 2^m$,m为类别的个数，对于两个类别来说，最大值为1

Information Gain 属性选择度量 之 信息增益

- 信息增益计算方式
- 信息增益
  - ✓ 用属性A划分样本数据集S，划分前样本数据集的不纯程度（熵）和划分后样本数据集的不纯程度（熵）的差值

$$\text{Gain}(S, A) = \text{Entropy}(S) - \text{Entropy}_A(S)$$

  - ✓ 按属性A划分S后的样本子集的熵定义：假定属性A具有K个不同的取值，将S划分为k个样本子集{S_1，S_2，…，S_k}，则按属性A划分S后的样本子集的信息熵如下，其中$|S_i|$（i=1,2,…,k）为样本子集S_i中包含的样本数，$|S|$为样本集S中包含的样本数

$$\text{Entropy}_A(S) = \sum_{i=1}^{k} \frac{|S_i|}{|S|} \text{Entropy}(S_i)$$

  - ✓ 信息增益越大，说明使用属性A划分后的样本子集越纯，越有利于分类

Weather数据集

outlook	temperature	humidity	wind	play ball
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rain	mild	high	weak	yes
rain	cool	normal	weak	yes
rain	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rain	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rain	mild	high	strong	no

ID3算法分析

$$\text{Entropy}(S) = \text{Entropy}(p_1, p_2, \cdots, p_m) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

$$\text{Entropy}_A(S) = \sum_{i=1}^{k} \frac{|S_i|}{|S|} \text{Entropy}(S_i)$$

$$\text{Gain}(S, A) = \text{Entropy}(S) - \text{Entropy}_A(S)$$

- 优点
  - 理论清晰，方法简单，学习能力较强
- 缺点
  - 只能处理分类属性数据，无法处理连续型数据
  - 对测试属性的每个取值相应产生一个分支，且划分相应的数据样本集，这样的划分会导致产生很多小的子集。随着子集被划分的越来越小，划分过程将会由于子集规模过小造成的统计特征不充分而停止
  - 使用信息增益作为决策树节点属性选择的标准。由于信息增益在类别值多的属性上计算结果大于类别值少的属性上计算结果，这将导致决策树算法偏向选择具有较多分支的属性，因而会造成过度拟合
    - 考虑充当唯一标识符的属性，如*product_ID*，*在该属性上划分* 将导致大量分区（与值一样多），每个只包含一个元组。由于每个分区都是纯的，所以基于该划分对数据集S分类的信息熵为0，通过对该属性的划分得到的信息增益最大。但显然，这种划分对分类没有用。

在创建树时如何选择属性
Measures of Impurity 不纯性度量



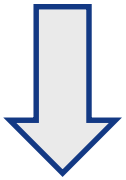
Gini Index



CART



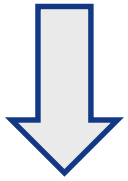
Entropy



ID3



Gain Ratio



C4.5

C4.5分类算法概述

- 基于ID3算法的不足，Quinlan在1993年对其做出改进，提出了改进的决策树分类算法C4.5
  - 能够处理连续型属性数据和离散型属性数据（ID3只能处理分类属性数据，无法处理连续型数据）
  - 能够处理具有缺失值的数据
  - 使用信息增益率作为决策树的属性选择标准
  - 对生成的树进行剪枝处理，以获取简略的决策树
  - 从决策树到规则的自动产生
- C4.5后续发展为商用的C5.0，在C4.5的基础上做了进一步改进
  - 使用提升技术，生成一系列决策树，然后集体投票决定分类结果
  - 支持新的数据类型，如日期等
  - 规则集没有顺序，而是所有匹配规则进行投票，更易解释

C4.5分类算法概念描述

— 信息熵（entropy）：用于度量一个属性的信息量

✓ 假定S为训练集，S的目标属性C具有m个可能的类标号值，$C=\{C_1,C_2,\ldots,C_m\}$，假定训练集S中C_i在所有样本中出现的概率为$p_i(i=1,2,3,\ldots,m)$，则该训练集S所包含的信息熵定义为

$$\text{Entropy}(S)=\text{Entropy}(p_1,p_2,\cdots,p_m)=-\sum_{i=1}^{m}p_i\log_2(p_i)$$

— 按属性A划分S后的样本子集的熵定义：

✓ 假定属性A为离散型数据，具有K个不同的取值，将S划分为k个样本子集{S1，S2，…，Sk}，则按属性A划分S后的样本子集的信息熵如下，其中|Si|（i=1,2,…,k）为样本子集Si中包含的样本数，|S|为样本集S中包含的样本数

$$\text{Entropy}_A(S)=\sum_{i=1}^{k}\frac{|S_i|}{|S|}\text{Entropy}(S_i)$$

✓ 假定属性A为连续型数据，则按属性A的取值递增排序，将每对相邻值的中点看做可能的分裂点，对每个可能的分裂点，计算信息熵。SL和SR对应于分裂点划分的左右两部分子集，选择信息熵最小点作为分裂点

$$\text{Entropy}_A(S)=\frac{|S_L|}{|S|}\text{Entropy}(S_L)+\frac{|S_R|}{|S|}\text{Entropy}(S_R)$$

C4.5分类算法概念描述

— 信息增益率

- ✓ 不仅考虑信息增益的大小程度，还兼顾考虑获得信息增益所付出的“代价”
- ✓ 引入属性的分裂信息来调节信息增益，如果某个属性有较多的分类取值，则它的信息增益会偏大，但信息增益率由于考虑了分裂信息而降低，进而消除了属性取值数目所带来的影响

$$\mathrm{SplitE}(A) = -\sum_{i=1}^{k} \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

$$\mathrm{GainRatio}(A) = \frac{\mathrm{Gain}(A)}{\mathrm{SplitE}(A)}$$

$$\mathrm{Gain}(S, A) = \mathrm{Entropy}(S) - \mathrm{Entropy}_A(S)$$

C4.5算法分析

- 优点
  - —产生的决策树简单直观
  - —产生的分类规则易于解释和应用
  - —分类准确率高
- 缺点
  - —在构造决策树过程中，需要对数据集进行多次顺序扫描和排序，导致算法低效

经典决策树算法的比较

不同决策树算法的比较
CART、ID3、c4.5

- 属性选择度量方法不同
  - — 信息增益
  - — 信息增益率
  - — 基尼指数和差异性损失
- 防止过分适应数据 的方法不同
  - —先剪枝
  - —后剪枝
- 适应的预测变量和目标变量的属性类型不同
  - —标称属性
  - —数值型数据
  - —缺失数据…



决策树算法的改进

大型数据库的分类挖掘 —— 可伸缩性

- 分类挖掘是一个在统计学和机器学习的领域也被广为研究的问题，并提出了很多算法，但是这些算法都是 **内存驻留** 的

- **可伸缩性问题：**

  由于训练元组在主存和高速缓存之间换进换出，决策树的构造可能变得效率低下。

  因此，要求以合理的速度对数以百万计的样本和数以百计的属性的进行分类挖掘。

- 解决方法：由大型数据库构造判定树
  - 首先将样本划分为子集，每个子集可以放在内存中
  - 然后由每个子集构造一颗决策树
  - 输出的分类法将每个子集的分类法组合在一起
  - （其他方法包括SLIQ, SPRINT,RainForest等等）

Decision Tree Classifier

— Overview

— Basic Algorithm to Build a Decision Tree

— Methods for Expressing Attribute Test Conditions

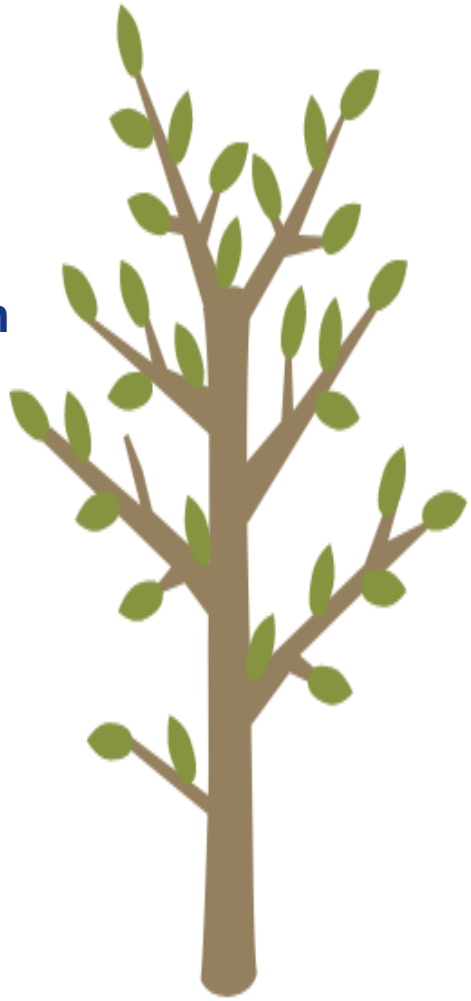
— Measures for Selecting an Attribute Test Condition

— Algorithm for Decision Tree Induction

— Classic decision tree algorithm

— Example Application: Web Robot Detection

— Characteristics of Decision Tree Classifiers



Example Application
Web Robot DetectionWeb 机器人检测

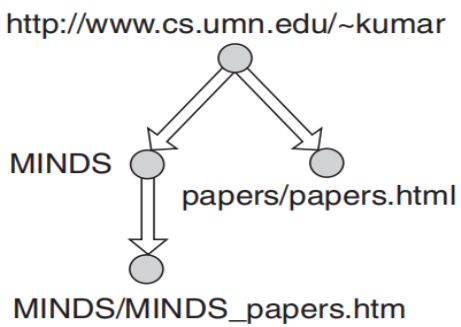
- 区分 Web 机器人的访问模式与人类用户的访问模式的任务
  - —Web 机器人(也称网络爬虫)是一种软件程序，通过跟踪从最初的一组种子 URL 中提取的超链接，从一个或多个网站自动检索文件。这些程序被应用于各种用途，从代替搜索引擎搜集Web 网页到执行一些更恶意的活动，如制造垃圾邮件、在线广告中制造点击欺诈……
  - —Web 机器人检测问题可以转换为二分类任务

Example Application
Web Robot DetectionWeb 机器人检测

- 分类任务的输入数据是一个Web 服务器日志，图显示了一个样例
  - 日志文件中的每一行对应于客户端(即人类用户或 Web 机器人)对 Web 服务器所做的请求。Web日志中记录的字段包括客户端的 IP 地址、 请求的时间戳、请求文件的 URL、文件的大小以及用户代理

Session	IP Address	Timestamp	Request Method	Requested Web Page	Protocol	Status	Number of Bytes	Referrer	User Agent
1	160.11.11.11	08/Aug/2004 10:15:21	GET	http://www.cs.umn.edu/~kumar	HTTP/1.1	200	6424		Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:15:34	GET	http://www.cs.umn.edu/~kumar/MINDS	HTTP/1.1	200	41378	http://www.cs.umn.edu/~kumar	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:15:41	GET	http://www.cs.umn.edu/~kumar/MINDS/MINDS_papers.htm	HTTP/1.1	200	1018516	http://www.cs.umn.edu/~kumar/MINDS	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:16:11	GET	http://www.cs.umn.edu/~kumar/papers/papers.html	HTTP/1.1	200	7463	http://www.cs.umn.edu/~kumar	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
2	35.9.2.2	08/Aug/2004 10:16:15	GET	http://www.cs.umn.edu/~steinbac	HTTP/1.0	200	3149		Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.7) Gecko/20040616

(a) Example of a Web server log.



(b) Graph of a Web session.

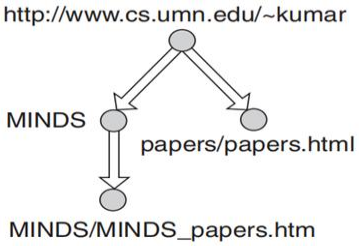
属性名称	描述
totalPages	一次Web会话提取的页面总数
ImagePages	一次Web会话提取的图像总数
TotalTime	网站访问者所用的时间
RepeatedAccess	一次Web会话多次请求同一页面
ErrorRequest	请求网页的错误
GET	使用GET方式提出的请求百分比
POST	使用POST方式提出的请求百分比
HEAD	使用HEAD方式提出的请求百分比
Breadth	Web遍历的宽度
Depth	Web遍历的深度
MultilP	使用多个IP地址的会话
MultiAgent	使用多个代理的会话

c)Web 机器人检测的导出属性

Example Application
Web Robot DetectionWeb 机器人检测

- 用户代理是包含有关客户端标识信息的字段。对于人类用户，用户代理字段指定用于提取文件的Web 浏览器或移动设备的类型，而对于Web 机器人，它应在技术上包含爬虫程序的名称
  - Web机器人可能会通过声明与已知浏览器相同的用户代理字段来隐藏其真实身份。因此，用户代理不是检测 Web 机器人的可靠方式
- 构建分类模型的第一步是精确定义数据实例和关联属性。一种简单的方法是将每个日志条目视为数据实例，并将日志文件中的相应字段用作其属性集，但这种方法由于几个原因而不够可靠
  - 首先，许多属性是标称值，并且取值范围广泛。例如，日志文件中唯一的客户端IP地 址、URL 和引用链接的数量可能非常大。这些属性对于构建决策树是不可取的，因为它们的划分信息非常大
  - 另外，可能无法对包含的 IP 地址、URL 或参考者的测试实例进行分类，这些实例不存在于训练数据中
  - 最后，通过将每个日志条目视为一个单独的数据实例，忽略了客户端检索的网页序列这一关键信息，可以帮助区分 Web 机器人访问与人类用户的访问

Example Application
Web Robot DetectionWeb 机器人检测



(b) Graph of a Web session.

- 将每个Web 会话视为数据实例是一种更好的选择
  - Web 会话是客户访问网站期间发出的一系列请求，每个Web 会话都可以模拟为有向图，其中结点对应于网页，边对应于将一个网页连接到另一个网页的超链接
  - 图b显示了日志文件中给出的第1个 Web会话的图形表示。每个 Web 会话都可以使用一些关于含有歧视性信息图形的有意义的属性进行表示
  - 图c显示了从图中提取的一些属性，包括扎根于网站入口处的相应树的深度和宽度。例如，图b 所示的树的深度和宽度都等于2。图c显示的派生属性比日志文件中给出的原始属性包含更多信息，因为它们表示了客户端在网站上的行为。由于 Web 机器人(1级) 和人类用户(0级)的会话数量相等，因此使用该方法创建了一个包含2916个实例的数据集。其中10%的数据用于训练，其余90%用于测试。生成的决策树如图所示，该决策树在训练 集上的错误率为3.8%，在测试集上的错误率为 5.3%

属性名称	描述
totalPages	一次Web会话提取的页面总数
ImagePages	一次Web会话提取的图像总数
TotalTime	网站访问者所用的时间
RepeatedAccess	一次Web会话多次请求同一页面
ErrorRequest	请求网页的错误
GET	使用GET方式提出的请求百分比
POST	使用POST方式提出的请求百分比
HEAD	使用HEAD方式提出的请求百分比
Breadth	Web遍历的宽度
Depth	Web遍历的深度
MultiIP	使用多个IP地址的会话
MultiAgent	使用多个代理的会话

c)Web 机器人检测的导出属性

决策树

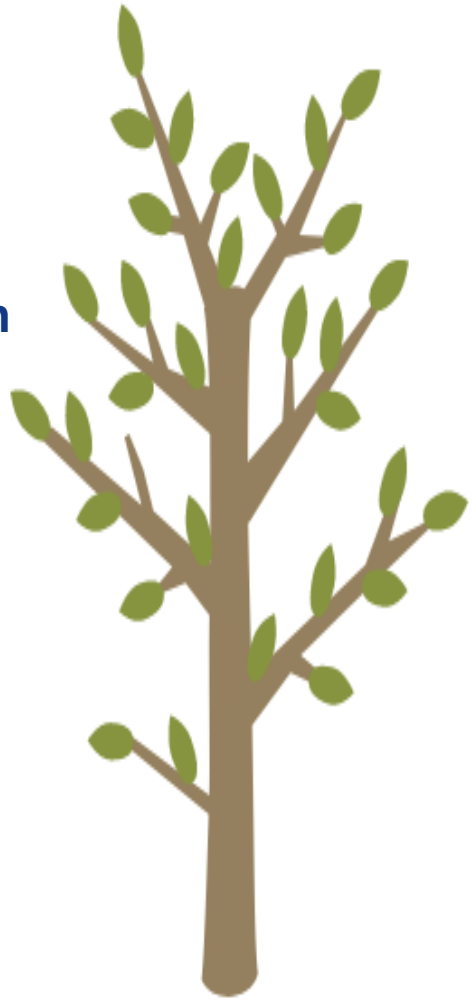
```
depth =1:
 breadth>7:class 1
 breadth<=7:
1I breadth <=3:
1|ImagePages>0.375:class 0
1 I ImagePages<=0.375:
 1 1 1 totalPages<=6:class 1
I 1 1 totalPages>6:
111I breadth <=1 :class 1
||11 breadth>1:class 0
 I width>3
1 1 MultiIP =0:
11|ImagePages<=0.1333: class 1
111 ImagePages>0.1333:
1 1 1 breadth<=6:class 0
1 I I breadth>6:class 1
11 1 MultiIP =1:
11|TotalTime <=361:class 0
111 1 TotalTime>361:class 1
depth>1:
I MultiAgent =0:
I depth>2:class 0
 1 depth<2:
1 I MultiIP =1 :class 0
| 1 MultiIP =0:
```

Example Application
Web Robot DetectionWeb 机器人检测

- 除了低错误率之外，决策树还显示了一些有趣的属性，有助于区分 Web 机器人与人类用户：
  - —1)Web 机器人的访问往往是广泛但浅显的，而人类用户的访问往往更集中(视野狭窄但深入)
  - —2)Web 机器人很少检索与网页关联的图像页面
  - —3)由Web 机器人产生的会话往往很长，并且包含大量请求页面
  - —4)由于人类用户检索的网页通常由浏览器缓存，所以 Web 机器人比人类用户更可能对同一网页重复请求

Decision Tree Classifier

- Overview
- Basic Algorithm to Build a Decision Tree
- Methods for Expressing Attribute Test Conditions
- Measures for Selecting an Attribute Test Condition
- Algorithm for Decision Tree Induction
- Classic decision tree algorithm
- Example Application: Web Robot Detection
- Characteristics of Decision Tree Classifiers

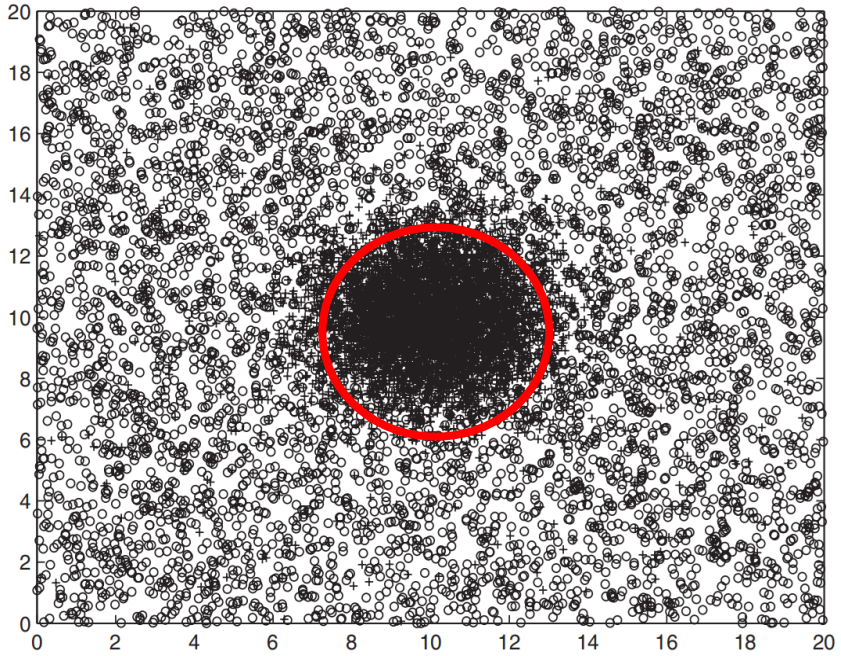


Chapter 7 Classification 分类
7-1: Basic Concepts 基本概念

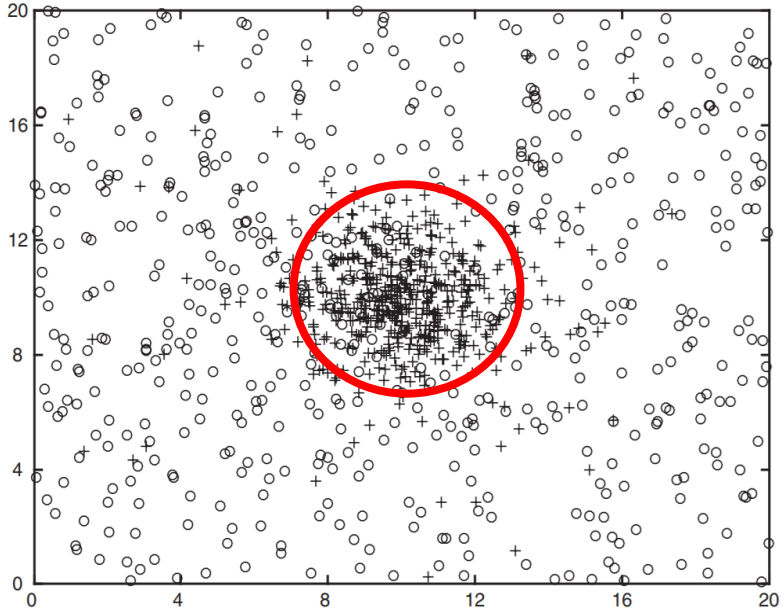
- Classification: Basic Concepts 基本概念
- Decision Tree Induction 决策树归纳
- **Model Overfitting 模型的过拟合**
- Model Selection 模型选择
- Model Evaluation 模型评估
- Presence of Hyper-parameters 超参数的使用
- Pitfalls of Model Selection and Evaluation 模型选择与评估中的陷阱
- Model Comparison 模型比较
- Techniques to Improve Classification Accuracy 提高分类准确率的技术

二维数据集例

- 图a 所示的二维数据集，实例属于两个类，标记为“+”和 “o”，每个类都有5400个实例。所有属于 “o” 类的实例服从均匀分布。在“+”类中，5000个实例的生成服从高斯分布，并通过单位方差法集中在(10,10)处，而其余400个实例的采样与 “o” 类一样服从均匀分布通过绘制以(10,10)为中心的适当大小的圆，可以很大程度地将“+”类与 “o” 类区分开来
- 为了生成这个二维数据集的分类器，随机抽取10%的数据进行训练，剩 余90%用于测试。图 b 所示的训练集看起来非常有代表性。使用基尼指数作为不 纯性度量来构造决策树，通过递归将结点扩展到子结点，直到每个叶结点都是纯的，以此 增加树的规模(叶结点数)



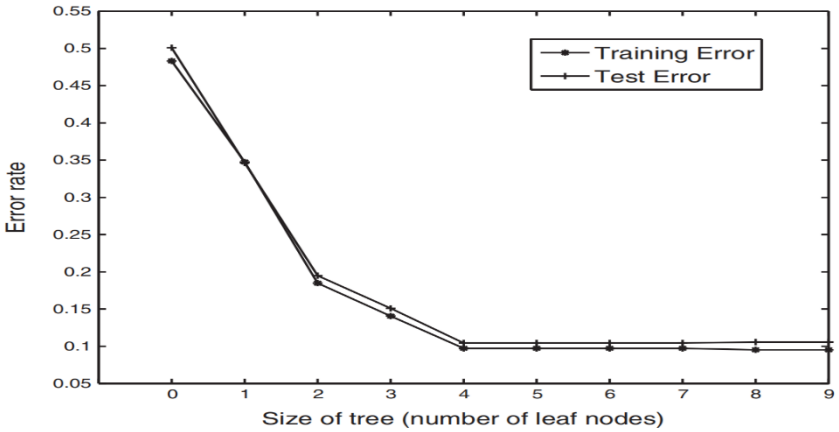
(a) Example of a 2-D data.



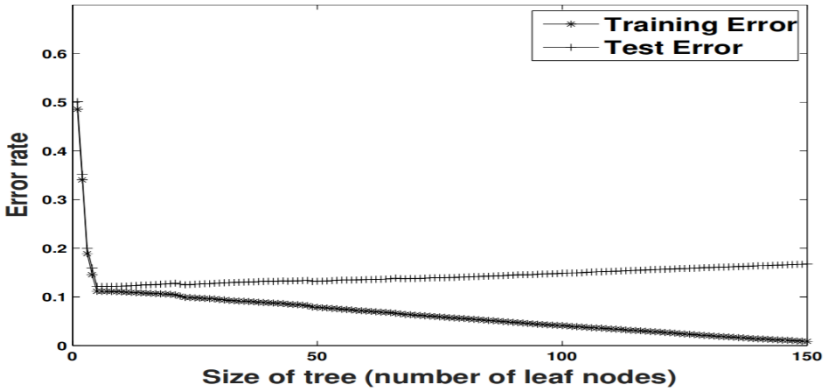
(b) Training set using 10% data.

模型的欠拟合(underfitting)和过拟合(overfitting)

- **图a显示树的大小从1到8变化时，训练集和测试集错误率的变化趋势。树最初只有一个或两个叶节点时，错误率都很高。这种情况称作模型欠拟合（model underfitting)**
  - 若学习的决策树过于简单，会导致欠拟合，以致无法充分表示属性与类别标签间的真实关系
- 随着将树的大小不断增大
  - 首先，由于较 大的树能够表示更复杂的决策边界，所以错误率会降低
  - 其次，训练集和测试集的错误率十分接近，这表明训练集在性能上足以代表泛化性能
  - 随着进 一 步将树的大小从8增加到 150,训练错误率继续稳定下降，直至最终达到零，如图b所示。然而，测试错误率在树的规模到达某 一 值时停止下降，之后开始增加。一 旦树的规模变得太大，训练错误率将不再足以评估测试集的错误率。此外，随着树的规模不断增大，训练错误率和测试错误率之间的差距不断扩大
- **这种看起来有悖直觉的现象被称为模型过拟合(model overfitting)。目前提出的分类模型学习方法试图在训练集上显示最低误差，但有的模型即使能够很好地覆盖训练数据，但它仍然可能表现出较差的泛化性能**



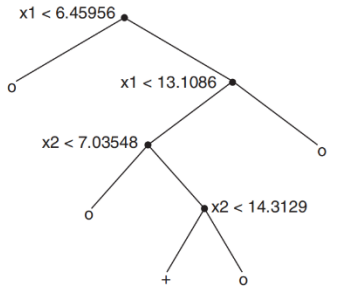
(a) Varying tree size from 1 to 8.



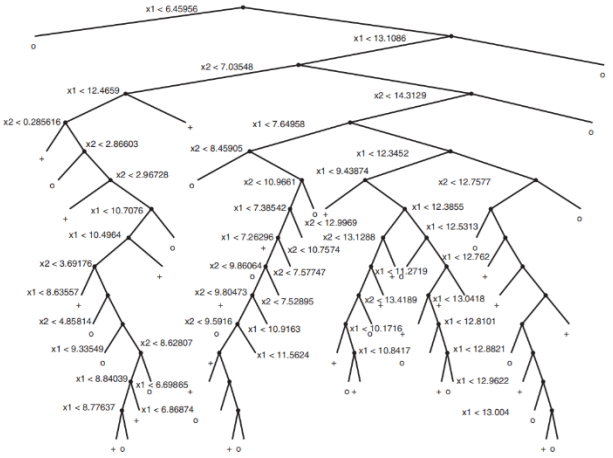
(b) Varying tree size from 1 to 150.

模型的欠拟合(underfitting)和过拟合(overfitting)

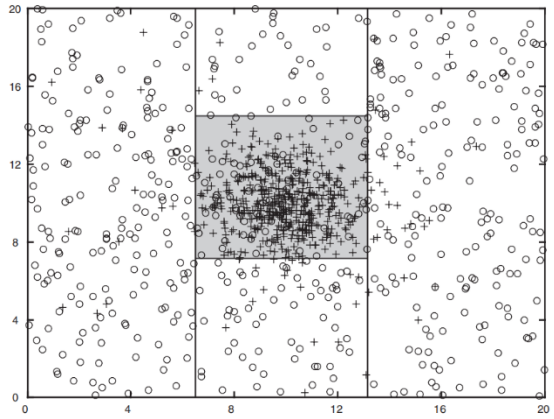
- 模型过拟合指的是在追求训练错误率最小化的过程中，选择了 一 种过于复杂的模型， 这种模型捕捉到了训练数据中的特定模式，但是没能获取到整体数据中属性和类别标签之间的本质关系

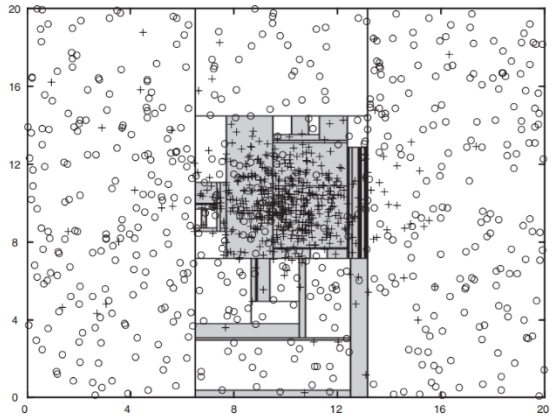


(a) Decision tree with 5 leaf nodes.



(b) Decision tree with 50 leaf nodes.





阴影矩形表示分配给“+”类的区域

- 规模为5的决策树非常简单，决策边界为最佳决策边界提供了一个合理的近似值，对应于以(10,10)处的高斯分布为中心的圆。虽然它的训练和测试错误率非零，但它们十分接近，表明在训练集中学习到的模式能很好地泛化到测试集
- 规模为50 的决策树更复杂，同时具有繁复的决策边界。例如，一些阴影矩形试图覆盖输入空间中仅包含一个或两个“+”类训练实例的狭窄区域。这类区域中的“+”实例在训练集中具有高度特异性，因为这些区域主要由整体数据集中的“一”实例占据。为了完美拟合训练数据，规模为50 的决 策树开始调整以适配测试集中的特定模式，导致在独立选择的测试集上性能较差

Chapter 7 Classification 分类
7-1: Basic Concepts 基本概念

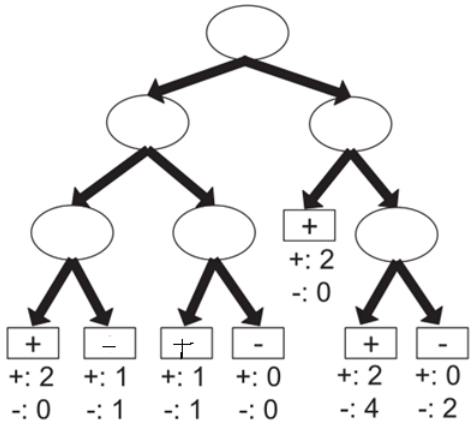
- Classification: Basic Concepts 基本概念
- Decision Tree Induction 决策树归纳
- Model Overfitting 模型的过拟合
- **Model Selection 模型选择**
- Model Evaluation 模型评估
- Presence of Hyper-parameters 超参数的使用
- Pitfalls of Model Selection and Evaluation 模型选择与评估中的陷阱
- Model Comparison 模型比较
- Techniques to Improve Classification Accuracy 提高分类准确率的技术

模型选择

- 模型选择(model selection)指的是选择出复杂度适合且可以很好地泛化到隐藏的测试实例的模型
  - —现有许多可行的分类模型，具有不同层次的模型复杂性，可用于捕获训练数据中的模式。在这些可选项中，需要选择的是泛化错误率最低的模型
  - —训练错误率不能可靠地作为模型选择的唯一标准，三种通用的方法来评估模型的泛化能力
    - 验证集应用
    - 模型复杂度合并
    - 统计范围估计
  - —主要讲述如何在决策树归纳中使用这些方法的特定策略

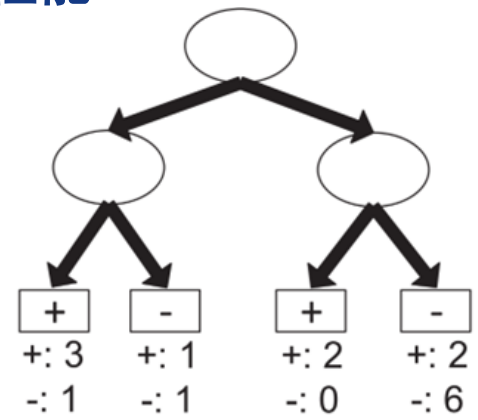
模型选择标准：验证集应用

- 通过“样本外”估计来评估模型的泛化错误率，在一个单独的验证集(validation set)上对模型进行评估，该集合未被用于模型训练
  - 由于验证集未被用于训练模型，因此验证集的错误率(称为验证错误率)比训练错误率更能说明泛化性能



Decision Tree, T_L

$$err_{val}(T_L) = 6/16 = 0.375$$



Decision Tree, T_R

$$err_{val}(T_R) = 4/16 = 0.25$$

- 给定一个训练集 D.train，将 D.train划分为两个更小子集 — D.tr和D.val。其中，D.tr用于训练，而D.val作为验证集
- 例如，将 D.train的2/3作为 D.tr 用于训练，剩余1/3作为 D.val计算验证错误率
- 对于任何经过 D.tr训练的分类模型 m，可以通过D.val 评估它的验证错误率——$err_{val}(m)$，其值最小的模型即为首选模型

模型选择标准：模型复杂度合并

- 由于模型越复杂，发生过拟合的可能性越大，因此模型选择方法**不仅要考虑训练错误率，还要考虑模型的复杂度**
  - 策略源于**奥卡姆剃刀原理**，如果两个模型具有相同误差，较简单模型将优于较复杂模型

奥卡姆剃刀原理：14世纪的逻辑学家、圣方济各会修士奥卡姆的威廉提出，其核心思想是“如无必要，勿增实体”，也被称为“简单有效原理”这个原理主张在解释自然现象或事物时，应优先考虑简单性，避免引入不必要的复杂性。当面对多种解释时，应优先选择最简单的那个。这一原则在科学和哲学领域有着广泛的影响，例如，在科学研究中，科学家们倾向于使用最简单的理论来解释自然现象

- 给定一个训练集 **D.train**，学习一个属于模型 **M** 的分类模型 **m** 。如果 **M** 代表所有可能的决策树集合，**m** 对应一个从训练集中学习到的特定决策树，**m** 的泛化错误率 **gen.error(m)** 。当模型复杂度较高时， **m** 的训练错误率 **train.error(m,D.train)** 会低估 **gen.error(m)** 。 因此，在使用训练错误率的同时，还使用模型 **M** 的复 杂 度 **complexity(M)** 来表示 **gen.error(m)** 函数：

$$gen.error(m)=train.error(m,D.train)+\alpha\times complexity(M)$$

- **α**是一个超参数，用于协调最小化训练错误和降低模型复杂度。 一个过大的**α**取值在评估过程中更强调模型复杂度对泛化性能的影响。可以使用利用验证集的方法选择合适的**α**值。例如，可以遍历**α**的所有可取值，从训练集的一个子集 **D.tr** 中学习到一个模型，然后通过一个独立的子集 **D.val** 计算得到模型的验证错误率，最后选择使验证错误率最低的**α**取值
- 上式提供了一种可行的方法，将模型复杂度纳入对泛化性能的评估中，该方法成为许多用于评估泛化性能的技术的核心，例如结构风险最小化原则、赤池信息准则 **(Akaike's Information Criterion,AIC)**、 贝叶斯信息准则**(Bayesian Information Criterion,BIC)**。 结构风险最小化原则是学习支持向量机的基础

Chapter 7 Classification 分类
7-1: Basic Concepts 基本概念

- Classification: Basic Concepts 基本概念
- Decision Tree Induction 决策树归纳
- Model Overfitting 模型的过拟合
- Model Selection 模型选择
- **Model Evaluation 模型评估**
- Presence of Hyper-parameters 超参数的使用
- Pitfalls of Model Selection and Evaluation 模型选择与评估中的陷阱
- Model Comparison 模型比较
- Techniques to Improve Classification Accuracy 提高分类准确率的技术

模型评估（Model Evaluation）

- 模型评估(model evaluation) 讨论估计模型泛化性能的方法，即D.train之外的未知实例的性能
  - 模型选择方法用于从训练集 D.train 中学习分类模型，也使用训练集D.train计算泛化性能的估计值
  - 然而，这些估计值是未知实例性能的偏向指标，因为它们被用来指导分类模型的选择。例如，如果使用验证错误率进行模型选择，会特意选择能得到最少验证集错误的模型作为结果。因此，验证错误率可能会低估真正的泛化错误率，不能可靠地用于模型评估
- 模型评估的正确方法是在被标注的测试集上，对学习模型性能进行评估，且这些测试集未在模型选择的任何阶段被使用
  - 这可以通过将整个被标记的实例集合D 划分为两个不相交的子集来实现， D.train用于模型选择，而 D.test 用于计算测试错误率err_{test}

Model Evaluation and Selection
模型评估与选择

- 评估度量：我们如何衡量准确率？其他的度量方法呢？
- 当评估准确率时使用**验证测试集**而不是训练集中带有类标签的元组
- 用于评估一个分类器的准确性的方法：
  - —保持和随机子抽样
  - —交叉验证
  - —自助方法
- 比较分类器：
  - —置信区间
  - —ROC曲线和成本效益分析

模型评估与选择

- 正元组：感兴趣的主要类的元组
- 负元组：其他元组
- 真阳性TP
  - —被分类器正确分类的正元组
- 真阴性TN
  - —被分类器正确分类的负元组
- 假阳性FP
  - —被错误地标记为正元组的负元组
- 假阴性FN
  - —被错误地标记为负元组的正元组
- TP和TN告诉我们分类器何时分类正确
- FP和FN告诉我们分类器何时错误

混淆矩阵

- 混淆矩阵：一个给定m(m>=2)个类，至少为m*m的表

Actual class\Predicted class	C_1	¬ C_1
C_1	**True Positives (TP)**	**False Negatives (FN)**
¬ C_1	**False Positives (FP)**	**True Negatives (TN)**

- 一个混淆矩阵样例：

Actual class\Predicted class	buy_computer = yes	buy_computer = no	Total
buy_computer = yes	**6954**	**46**	7000
buy_computer = no	**412**	**2588**	3000
Total	7366	2634	10000

- CMi,j指出类i的元组被分类器标记为类j的个数
- 可能需要多余的列和行来提供合计

准确率，错误率，灵敏性和特效性

A\P	C	¬C	
C	TP	FN	P
¬C	FP	TN	N
	P’	N’	All

- 分类器准确率: 正确分类元组所占的百分比

  Accuracy = (TP + TN)/All

- **Error rate: *1 – accuracy*, or**

  Error rate = (FP + FN)/All

- 类不平衡问题:
  - 感兴趣的主类是稀少的, **e.g.** 欺诈
  - 负类显著地占多数，正类占少数
  - 灵敏性: 正确识别正元组的百分比
    - **Sensitivity = TP/P**
  - 特效性:正确识别负元组的百分比
    - **Specificity = TN/N**

精度和召回率

- 精度： 标记为正类的元组实际为正类的百分比

$$precision = \frac{TP}{TP + FP}$$

- 召回率：正元组标记为正的百分比
- 满分为1.0

$$recall = \frac{TP}{TP + FN}$$

- 将精度和召回率组合到一个度量中：
  - *F* 度量 (*F*$_1$ or *F*-分数):精度和召回率的调和平均数

$$F = \frac{2 \times precision \times recall}{precision + recall}$$

- *F*$_\beta$:精度和召回率的加权度量

$$F_\beta = \frac{(1 + \beta^2) \times precision \times recall}{\beta^2 \times precision + recall},$$

样例

Actual Class\Predicted class	cancer = yes	cancer = no	Total	Recognition(%)
cancer = yes	**90**	**210**	300	30.00 (*sensitivity*
cancer = no	**140**	**9560**	9700	98.56 (*specificity)*
Total	230	9770	10000	96.40 (*accuracy*)

- 精度： 标记为正类的元组实际为正类的百分比
- 召回率：正元组标记为正的百分比

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

样例

Actual Class\Predicted class	cancer = yes	cancer = no	Total	Recognition(%)
cancer = yes	**90**	**210**	300	30.00 (*sensitivity*
cancer = no	**140**	**9560**	9700	98.56 (*specificity)*
Total	230	9770	10000	96.40 (*accuracy*)

- 精度： 标记为正类的元组实际为正类的百分比
- 召回率：正元组标记为正的百分比

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

- ***Precision* = 90/230 = 39.13%**
- ***Recall* = 90/300 = 30.00%**

使用统计显著性检验选择模型

- 假设我们有两个分类器，M1和M2，哪一个更好？
- 直观的，可以选择最低错误率的模型
- 然而
  - —10-折交叉验证实验的错误率之间存在很大的方差
  - —平均错误率可能不同，但差别不是统计显著的

使用统计显著性检验选择模型 Cont.

- 10次10-折交叉验证，每次使用数据不同的10折划分
- 每个划分都独立的抽取
- 每个错误率都可以看做来自一种概率分布不同的独立样本
- 假设这些错误率服从具有k-1个自由度的t分布，其中k等于10
- 使用t-检验

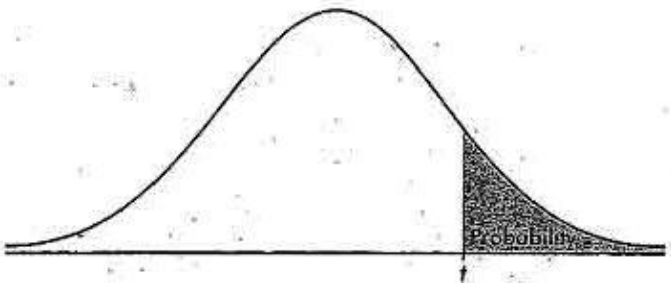
$$t = \frac{\overline{err}(M_1) - \overline{err}(M_2)}{\sqrt{var(M_1 - M_2)/k}}$$

- 单检验集

$$var(M_1 - M_2) = \frac{1}{k}\sum_{i=1}^{k}\left[err(M_1)_i - err(M_2)_i - (\overline{err}(M_1) - \overline{err}(M_2))\right]^2$$

- 假设两者模型相同（平均错误率之差为0）
- 若拒绝假设，那么差是统计显著的
  - 选择具有较低错误率的模型

使用统计显著性检验选择模型 Cont.



- 对称的
- 显著水平, e.g., *sig = 0.05* or *5%* *意味着* M_1 & M_2 的差对总体的95%显著不同
- 置信界, *z = sig/2*
- ***t>z或t<-z，可以拒绝原假设，有着统计显著的差别***

TABLE B: *t*-DISTRIBUTION CRITICAL VALUES

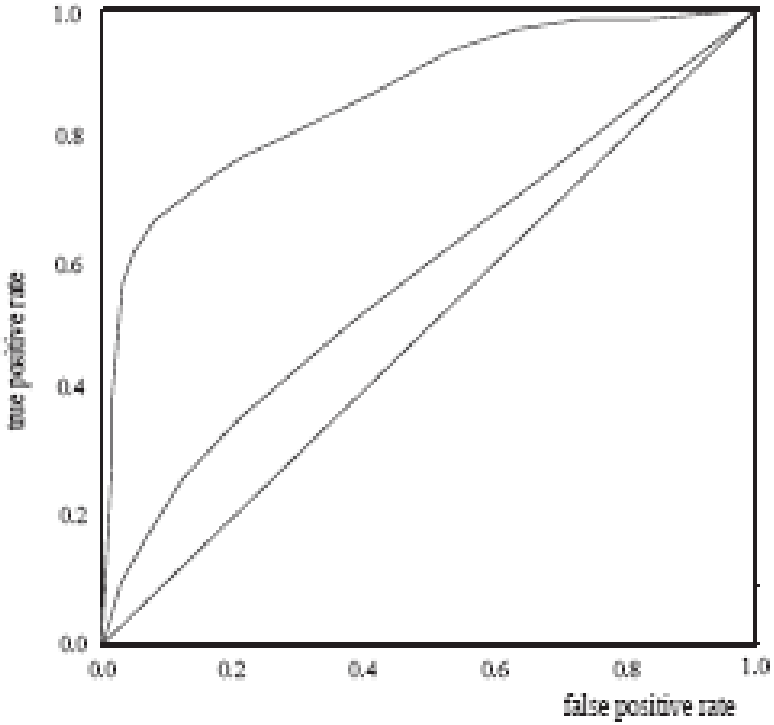
	Tail probability *p*											
df	.25	.20	.15	.10	.05	.025	.02	.01	.005	.0025	.001	.0005
1	1.000	1.376	1.963	3.078	6.314	12.71	15.89	31.82	63.66	127.3	318.3	636.6
2	.816	1.061	1.386	1.886	2.920	4.303	4.849	6.965	9.925	14.09	22.33	31.60
3	.765	.978	1.250	1.638	2.353	3.182	3.482	4.541	5.841	7.453	10.21	12.92
4	.741	.941	1.190	1.533	2.132	2.776	2.999	3.747	4.604	5.598	7.173	8.610
5	.727	.920	1.156	1.476	2.015	2.571	2.757	3.365	4.032	4.773	5.893	6.869
6	.718	.906	1.134	1.440	1.943	2.447	2.612	3.143	3.707	4.317	5.208	5.959
7	.711	.896	1.119	1.415	1.895	2.365	2.517	2.998	3.499	4.029	4.785	5.408
8	.706	.889	1.108	1.397	1.860	2.306	2.449	2.896	3.355	3.833	4.501	5.041
9	.703	.883	1.100	1.383	1.833	2.262	2.398	2.821	3.250	3.690	4.297	4.781
10	.700	.879	1.093	1.372	1.812	2.228	2.359	2.764	3.169	3.581	4.144	4.587
11	.697	.876	1.088	1.363	1.796	2.201	2.328	2.718	3.106	3.497	4.025	4.437
12	.695	.873	1.083	1.356	1.782	2.179	2.303	2.681	3.055	3.428	3.930	4.318
13	.694	.870	1.079	1.350	1.771	2.160	2.282	2.650	3.012	3.372	3.852	4.221
14	.692	.868	1.076	1.345	1.761	2.145	2.264	2.624	2.977	3.326	3.787	4.140
15	.691	.866	1.074	1.341	1.753	2.131	2.249	2.602	2.947	3.286	3.733	4.073
16	.690	.865	1.071	1.337	1.746	2.120	2.235	2.583	2.921	3.252	3.686	4.015
17	.689	.863	1.069	1.333	1.740	2.110	2.224	2.567	2.898	3.222	3.646	3.965
18	.688	.862	1.067	1.330	1.734	2.101	2.214	2.552	2.878	3.197	3.611	3.922
19	.688	.861	1.066	1.328	1.729	2.093	2.205	2.539	2.861	3.174	3.579	3.883
20	.687	.860	1.064	1.325	1.725	2.086	2.197	2.528	2.845	3.153	3.552	3.850
21	.686	.859	1.063	1.323	1.721	2.080	2.189	2.518	2.831	3.135	3.527	3.819
22	.686	.858	1.061	1.321	1.717	2.074	2.183	2.508	2.819	3.119	3.505	3.792
23	.685	.858	1.060	1.319	1.714	2.069	2.177	2.500	2.807	3.104	3.485	3.768
24	.685	.857	1.059	1.318	1.711	2.064	2.172	2.492	2.797	3.091	3.467	3.745
25	.684	.856	1.058	1.316	1.708	2.060	2.167	2.485	2.787	3.078	3.450	3.725
26	.684	.856	1.058	1.315	1.706	2.056	2.162	2.479	2.779	3.067	3.435	3.707
27	.684	.855	1.057	1.314	1.703	2.052	2.158	2.473	2.771	3.057	3.421	3.690
28	.683	.855	1.056	1.313	1.701	2.048	2.154	2.467	2.763	3.047	3.408	3.674
29	.683	.854	1.055	1.311	1.699	2.045	2.150	2.462	2.756	3.038	3.396	3.659
30	.683	.854	1.055	1.310	1.697	2.042	2.147	2.457	2.750	3.030	3.385	3.646
40	.681	.851	1.050	1.303	1.684	2.021	2.123	2.423	2.704	2.971	3.307	3.551
50	.679	.849	1.047	1.299	1.676	2.009	2.109	2.403	2.678	2.937	3.261	3.496
60	.679	.848	1.045	1.296	1.671	2.000	2.099	2.390	2.660	2.915	3.232	3.460
80	.678	.846	1.043	1.292	1.664	1.990	2.088	2.374	2.639	2.887	3.195	3.416
100	.677	.845	1.042	1.290	1.660	1.984	2.081	2.364	2.626	2.871	3.174	3.390
1000	.675	.842	1.037	1.282	1.646	1.962	2.056	2.330	2.581	2.813	3.098	3.300
∞	.674	.841	1.036	1.282	1.645	1.960	2.054	2.326	2.576	2.807	3.091	3.291
	50%	60%	70%	80%	90%	95%	96%	98%	99%	99.5%	99.8%	99.9%

Confidence level *C*

模型评估与选择 —— ROC 曲线

- ROC（Receiver Operating Characteristic）曲线
  - 显示了给定模型的真正利率(TPR)和假正利率(FPR)之间的权衡
  - ROC 曲线离对角线越近

    模型的准确率越低
    - 好的模型的 ROC 曲线
  - 曲线下方的面积 -> 模型准确率
    - 面积越接近0.5，准确率越低



  - 横坐标：FPR 假正利率
  - 纵坐标：TPR 真正利率

Holdout Methods

模型评估与选择 —— 1) 保持 & 随机二次抽样

- **Holdout Methods** 保持方法
  - 给定数据随机地划分为两个独立的集合：训练集(E.g., 2/3)和 检验集(E.g., 1/3)
  - 使用 **训练集** 导出模型，使用 **检验集** 估计其准确率（估计是悲观的）

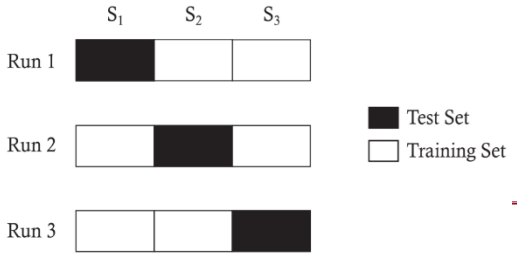
- **Random sampling** 随机二次抽样
  - 保持方法的一种变形
  - 将保持方法重复 k 次，总准确率估计取每次迭代准确率的平均值（avg）

模型评估：（1）保持方法

- 把 D 划分 D.train和 D.test的两种不同方法，计算测试错误率err_{test}
  - 保持方法（Holdout Method）
  - 交叉验证（Cross-Validation）
- 保持方法中有标记数据集合D 被随机划分为两个不相交的集合，即训练集D.train和测试集 D.test。 然后使用模型选择方法从 D.train归纳得到分类模型，并用其在 D.test 中的错误率err_{test}估计泛化错误率
- 用于训练和测试的数据保留比例主要由分析师决定。例如，三分之二用于训练，三分之一用于测试。类似于将 D.train 划分为 D.tr 和 D.val 时所面临的权衡，在标记数据集中，选择用于训练和测试的正确比例是十分重要的
  - 如果 D.train 的规模很小，使用数量不足的训练实例可能学习到错误的分类模型，从而导致对泛化性能的估计有偏差
  - 如果 D.test 的规模很小，那么 err_{test} 可能不太可靠，因为是基于少量测试实例计算出的
  - 当将D 随机划分为D.train 和 D.test 时， err_{test}可能会有很大的变化。保持方法可以多次重复后获得测试错误率的分布，这种方法称为随机子采样(random subsampling) 或重复保持方法，该方法生成的错误率分布有助于理解 err_{test} 的方差

模型评估：（2）交叉验证

- 把 D 划分 D.train和 D.test的两种不同方法，计算测试错误率err_{test}
  - 保持方法（Holdout Method）
  - 交叉验证（Cross-Validation）
- 交叉验证旨在有效地利用D中所有的标记实例进行训练和测试。假设给定一 个有标记的集合，将其随机地划分为三个相同大小的子集S_1、S_2 和 S_3
  - 第一次运行，使用子集 S_2 和 S_3 训练模型(显示为空白块)，并在子集S_1 上测试模型的测试错误率$err(S_1)$
  - 第二次运行，使用S_1 和 S_3 作为训练集， S_2 作为测试集，并计算测试错误率$err(S_2)$
  - 第三次运行，使用S_1 和 S_2 作 为训练集，S_3 作为测试集，并计算测试错误率$err(S_3)$
  - 总的测试错误率由所有运行中的子集测试错误率之和除以总的实例数得到，这种方法被称为三重交叉验证
- k重交叉验证方法指将标记数据D (大小为N) 分割成k个相等大小的分区(或子类)的方法
  - 第 i 次运行，D— 个分区作为 D.test(i) 用于测试，其余的分区作为 D.train(i) 用于训练。使用 D.train(i) 学习得到模型m(i)，并在D.test(i) 上获得测试错误率之和 err m(i)
  - 该过程重复 k 次，数据集中的每个实例恰好用于一次测试、(k—1) 次训练
  - 总测试错误率err_{test}：



$$err_{test} = \frac{\sum_{i=1}^{k} err_{sum}(i)}{N}$$

模型评估：（2）交叉验证

- k 重交叉验证中 k 的正确选择取决于问题的若干特征
- 较小的 k 值使得每次运行时的训练集较小，这将导致得到的泛化错误率估计比在整个标记集上训练的模型的预期更大
- k 的取值过高会导致每次运行时的训练集过大，这降低了泛化错误率估计中的偏置。极端情况，k=N，每次运行只使用一个数据实例进行测 试，其余数据用于训练。这种 k 重交叉验证的特殊情况称为“留一法”(leave-one-out)。这种方法的优点是能尽可能多地利用训练数据。但是，留一法可能会在一些特殊情况下产生十分具有误导性的结果
- 对于大型数据集，留一法在计算上很昂贵，因为交叉验证程序需重复 N 次。对于大多数实际应用，k 在 5 到10 之间进行选择，可提供估计泛化错误率的合理方法，因为每次重复都能够利用80%到 90%的标记数据进行训练
- k 重交叉验证产生单一泛化错误率估计，不提供关于估计方差的任何信息。为了获得这些信息，可以对每种可能的 k 个数据分区运行 k 重交叉验证，以此根据每个分区的计算，获得测试错误率分布。使用所有可能的划分的平均测试误差，能得到更稳健的泛化错误率估计。这种估计泛化错误率及其方差的方法称为完全交叉验证(complete cross-validation)法。尽管这样的估计非常稳健，但当数据集规模很大时，要得到所有可能的 k 个分区，通常代价太过昂贵

Chapter 7 Classification 分类
7-1: Basic Concepts 基本概念

- Classification: Basic Concepts 基本概念
- Decision Tree Induction 决策树归纳
- Model Overfitting 模型的过拟合
- Model Selection 模型选择
- Model Evaluation 模型评估
- **Presence of Hyper-parameters 超参数的使用**
- Pitfalls of Model Selection and Evaluation 模型选择与评估中的陷阱
- Model Comparison 模型比较
- Techniques to Improve Classification Accuracy 提高分类准确率的技术

超参数的使用(PresenceofHyper-parameters)

- 超参数是学习算法的参数，需要在学习分类模型之前确定
  - —例：超参数α用于估计模型选择方法的泛化错误率中，用于明确表示模型的复杂性

    gen.error(m)=train.error(m,D.train)+ α ×complexity(M)
- 与常规模型参数(例如决策树内部结点中的测试条件)不同，超参数(例如α)不会出现在用于对未标记样本进行分类的最终分类模型中
  - —超参数的取值需要在模型选择期间被确定，该过程称为超参数选择(hyper-parameter selection)
  - —模型评估期间也需要考虑超参数
- 稍微修改前一节描述的交叉验证方法，就可以有效地完成这两项任务。

Chapter 7 Classification 分类
7-1: Basic Concepts 基本概念

- Classification: Basic Concepts 基本概念
- Decision Tree Induction 决策树归纳
- Model Overfitting 模型的过拟合
- Model Selection 模型选择
- Model Evaluation 模型评估
- Presence of Hyper-parameters 超参数的使用
- **Pitfalls of Model Selection and Evaluation 模型选择与评估中的陷阱**
- Model Comparison 模型比较
- Techniques to Improve Classification Accuracy 提高分类准确率的技术

模型选择与评估中的陷阱

- 有效地应用模型选择和评估，可作为学习分类模型以及评估其泛化性能的优秀工具。 然而，在实际环境中有效地使用它们时，存在一些会导致错误的或有误导性的结论的陷阱
  - 其中一些陷阱很容易理解，也很容易避免
  - 而其他一些陷阱本质上非常微妙，难以捕捉
  - 介绍其中的两个陷阱 ， 并讨论用于避免它们的最佳措施

（1）训练集和测试集之间的重叠

- 对于一个没有瑕疵的模型选择和评估方案，基本要求之一是用于模型选择的数据(D.train)必须与用于模型评估的数据(D.test)分开。如果两者之间存在重叠，在 D.test 上计算得到的测试错误率err_{test}不能代表未知实例的性能。使用err_{test}比较分类模型的有效性可能会产生误导，因为过于复杂的模型可能会由于模型过拟合而显示出错误的err_{test}低取值
- **为了说明确保 D.train 和 D.test 之间不重叠的重要性**，考虑属性都是不相关的标记数据集，即这些属性与类别标签没有关系。使用这些属性时，认为分类模型没有随机猜测的执行效果好
- 但是，如果测试集包含的数据实例(即使是少量的)用于训练，则过于复杂的模型的性能可能比随机分类器表现得更好，即使属性是完全无关的。这种情况实际可以作为评判标准，用于检测由于实验设置不当而导致的过拟 合。如果模型对不相关属性显示出比随机分类器更好的性能，它也能体现训练集和测试集之间的潜在反馈

模型选择与评估中的陷阱

（2）使用验证错误率作为泛化错误率

- 验证错误率err_{test}在模型选择过程中起着重要作用，因为它提供了模型在未用于模型训练的实例 D.val 上的“样本外”误差估计。因此，与用于模型选择和超参数取值的训练错误率相比， err_{test}是更好的度量标准
- 但是， 一旦验证集用于选择分类模型m*， err_{test}就不再能反映出m*在未知实例上的性能
- 为了明晰使用验证错误率作为泛化性能估计时的陷阱，考虑使用验证集 D.val， 从值域 P 中选择出取值为 p 的超参数的问题。如果 P 中的可取值数量非常大并且 D.val 的规模很小，则可以随机在 D.val 上选择出性能良好的超参数p*。 尽管使用p* 学习的分类模型m* 具有较低的验证错误率，但它缺乏在未知测试实例上的普适性
- 估计模型m* 的泛化错误率的正确方法是使用独立选择的测试集 D.test， 而该测试集未以任何方式用于影响m* 的选择。根据经验，在模型选择过程中不应检查测试集，以确保不存在任何形式的过拟合。如果从标记数据集的任意部分获得了可以改进分类模型的助力，即使是以间接的方式，那么在测试期间也必须丢弃该部分数据

Chapter 7 Classification 分类
7-1: Basic Concepts 基本概念

- Classification: Basic Concepts 基本概念
- Decision Tree Induction 决策树归纳
- Model Overfitting 模型的过拟合
- Model Selection 模型选择
- Model Evaluation 模型评估
- Presence of Hyper-parameters 超参数的使用
- Pitfalls of Model Selection and Evaluation 模型选择与评估中的陷阱
- **Model Comparison 模型比较**
- Techniques to Improve Classification Accuracy 提高分类准确率的技术

模型比较

- 比较不同分类模型性能时的一个困难是，观察到的性能差异是否具有统计显著性
  - 例如，考虑两个分类模型M_A 和 M_B。 假设在包含30个实例的测试集上进行评估时M_A的准确率为85%，而M_B在另一个包含5000个实例的测试集上的准确率为75%。基于这些信息， M_A是比M_B更好的模型吗?
- 此示例提出了两个关于性能指标统计显著性的关键问题：
  - 1)尽管M_A具有比M_B更高的准确率，但它是在较小的测试集上进行测试的。在多大程度上认为M_A实际上的准确率是85%?第一个问题涉及对模型准确率置信区间的估计
  - 2)是否有可能解释为，由于测试集的组成发生了变化，造成M_A和 M_B准确率之间的差异?第二个问题涉及对观测偏差统计显著性的测试

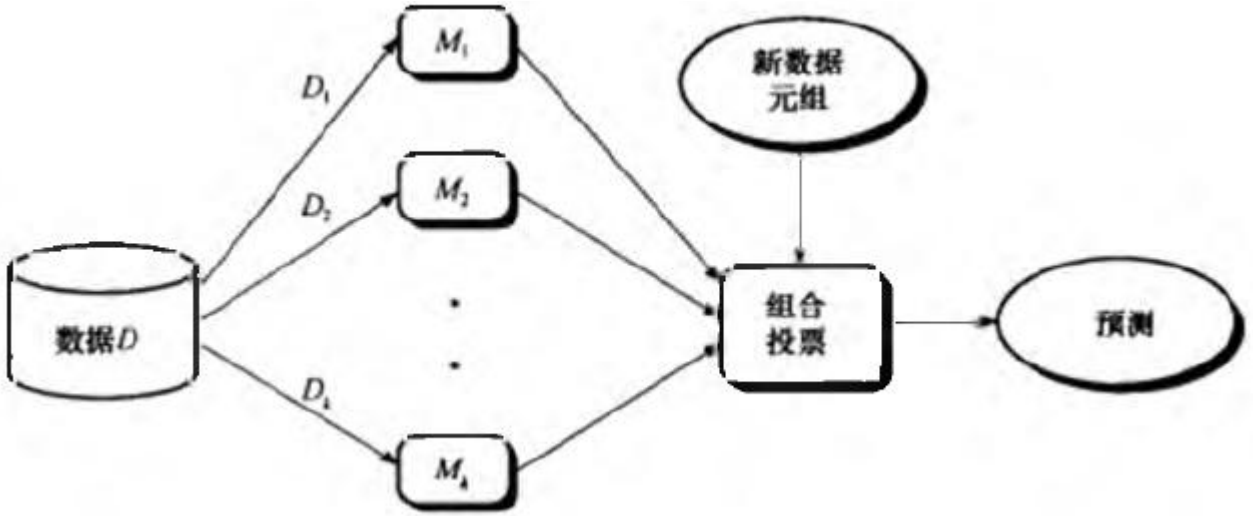
Chapter 7 Classification 分类
7-1: Basic Concepts 基本概念

- Classification: Basic Concepts 基本概念
- Decision Tree Induction 决策树归纳
- Model Overfitting 模型的过拟合
- Model Selection 模型选择
- Model Evaluation 模型评估
- Presence of Hyper-parameters 超参数的使用
- Pitfalls of Model Selection and Evaluation 模型选择与评估中的陷阱
- Model Comparison 模型比较
- **Techniques to Improve Classification Accuracy 提高分类准确率的技术**

Techniques to Improve Classification Accuracy
提高分类准确率的技术

- 通过组合分类器，基于个体分类器投票，返回最终的类标号预测
- 常见的组合方法：
  - Bagging 装袋
  - Boosting 提升
  - Random Forest 随机森林

提高模型的准确率：组合分类方法产生一系列分类模型 M_1，M_2，…，M_k。给定一个待分类的新数据元组，每个基分类器对该元组的类标号“投票”。组合分类器组合这些投票返回类预测



Bagging: Boostrap Aggregation

提高分类准确率的技术 —— 1) 装袋

- 多个分类器，每个分类器有着相同的投票权重
  - —多数表决
- d个元组的集合D
- 对于迭代i(i=1,2…k),d个元组的训练集Di采用有放回抽样
  - —有些元组不出现
  - —有些元组重复多次
- 每个训练集Di学习得到分类模型Mi
- 未知元组X分类
  - —每个分类器Mi返回它的类预测
  - —得票最高的类赋予X
- 可以用于连续值的预测

Bagging: Boostrap Aggregation

提高分类准确率的技术 —— 1) 装袋

Algorithm: Bagging. The bagging algorithm—create an ensemble of classification models for a learning scheme where each model gives an equally weighted prediction.

Input:

- D, a set of d training tuples;
- k, the number of models in the ensemble;
- a classification learning scheme (decision tree algorithm, naïve Bayesian, etc.).

Output: The ensemble—a composite model, $M*$.

Method:

(1) **for** $i = 1$ to k **do** // create k models:
(2) create bootstrap sample, D_i, by sampling D with replacement;
(3) use D_i and the learning scheme to derive a model, M_i;
(4) **endfor**

To use the ensemble to classify a tuple, X:

let each of the k models classify X and return the majority vote;

Boosting 提高分类准确率的技术 —— 2) 提升

- 基本思想：
  - 赋予每个训练元组以权重， 迭代地学习 k 个分类器 M_i
  - 更新每个训练元组的权重，再次生成新的分类器
  - 最终根据权重衡量各个子分类器的投票结果，得到最终的类

- **Adaboost：一种流行的提升算法**
  - 1) 给予所有训练元组相同的权重1/d
  - 2) 根据权重，有放回的抽样，导出k 个分类器 M_i
  - 3) 如果元组不正确地分类，则增加它的权重
    - 希望能更关注上一轮“误分类”的元组
    - 模型的错误率 （如果元组被误分类，则 err(Xj) 被置为1

$$error(M_i) = \sum_j^d w_j \times err(\mathbf{X_j})$$

  - 4) 各个子分类器的投票权重 为

$$\log \frac{1-error(M_i)}{error(M_i)}$$

  - 5) 具有最大权重和的类为“赢家”，并返回作为对元组 ***X*** 的类预测

Random Forest

提高分类准确率的技术 —— 3) 随机森林

- 想象组合分类器中的每个分类器都是一颗决策树
  因此分类器的集合就是一个“森林”

- 分类时，每棵树都投票并返回得票最多的类

- 随机森林可以使用 **装袋** 与 **随机属性选择** 结合来构建

- **与 Adaboost 相比**
  - 对错误和离群点鲁棒性更好